



$$P(A|B) = \frac{P(B|A) P(A)}{P(B)}$$

Diagram illustrating Bayes' Theorem with labels:

- Posterior Probability** points to $P(A|B)$.
- Likelihood** points to $P(B|A)$.
- Prior Probability** points to $P(A)$.
- $P(B)$ is the denominator.

Definitions & Assumptions

1. Naïve Bayes Classifier

Definition:

Naïve Bayes is a probabilistic classifier that applies Bayes' theorem with the simplifying assumption that **features are conditionally independent** given the class. It predicts the most likely class based on prior probabilities and likelihoods.

Assumptions:

- **Conditional Independence:** Each feature contributes independently to the probability once the class is known.
- **Correct Distributional Form:**
 - For categorical features → probabilities estimated from counts.
 - For continuous features → often assume Gaussian distribution.

- **Sufficient Data / No Zero Counts:** Need enough samples for each feature-class combination. Zero counts are fixed by smoothing (Laplace/add- α).
 - **Class Prior Knowledge:** Priors estimated correctly from training data or domain knowledge.
-

2. Bayes Optimal Classifier

Definition:

The Bayes optimal classifier is the theoretical best possible classifier. It chooses the class with the highest true posterior probability given the features. It minimizes the expected error rate.

Assumptions:

- **Perfect Knowledge:** Exact class priors and full class-conditional probability distributions are known.
 - **Well-defined Feature Space:** All possible feature combinations and their probabilities are included.
 - **Noise is accounted for:** Assumes stochastic variation is modeled correctly.
-

3. Logistic Regression

- The *logistic function* or *sigmoid function* is given as:

$$h(g(\mathbf{x})) = \frac{1}{1 + e^{-g(\mathbf{x})}}; \Rightarrow h(\mathbf{x}) = \frac{1}{1 + e^{-(\mathbf{w}^T \mathbf{x} + w_0)}}$$

$$p(y | \mathbf{x}, \mathbf{w}) = (h_{\mathbf{w}}(\mathbf{x}))^y (1 - h_{\mathbf{w}}(\mathbf{x}))^{1-y}$$

$$J(\mathbf{w}) = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log(h_{\mathbf{w}}(\mathbf{x}^{(i)})) + (1 - y^{(i)}) \log(1 - h_{\mathbf{w}}(\mathbf{x}^{(i)})) \right]$$

- **Odds:** The odds of an event represent the probability (p) that the event will occur compared to the probability that it will not occur ($1 - p$).

$$Odds = \frac{p}{1-p}$$

- For example, if the probability of an event (like winning a game) is 0.75 (or 75%), then the odds of winning are $0.75/0.25=30.75/0.25=3$.
- This means the event is 3 times more likely to occur than not.
- **Logit or Log-Odds:** The logit or log-odds is a function that transforms a probability p from the interval $(0, 1)$ to a real number $(-\infty, +\infty)$.
- The logits are calculated as:

$$logit(p) = \log\left(\frac{p}{1-p}\right)$$

- This transformation is particularly useful in logistic regression for linking the odds linearly with predictor variables.
- If p is the probability of an event, then the log-odds would be positive if $p > 0.5$, zero if $p = 0.5$, and negative if $p < 0.5$.

Definition:

Logistic regression is a discriminative model that estimates the probability of a binary outcome using the logistic (sigmoid) function applied to a linear combination of input features.

Assumptions:

- **Linearity in the Log-Odds:** The log-odds of the outcome is a linear function of input features.
- **Independent Observations:** Training examples are independent of each other.

- **No Perfect Multicollinearity:** Features should not be exact linear combinations of each other.
 - **Large Sample Size:** Maximum likelihood estimation works best with large datasets.
 - **Well-specified Model:** All relevant predictors included, no systematic omission.
-

4. Linear Regression / Curve Fitting

Definition:

Linear regression models the relationship between a dependent variable and one or more independent variables by fitting a linear equation to observed data.

Assumptions:

- **Linearity in Parameters:** Target variable is a linear combination of parameters (may use polynomial basis functions).
 - **Additive Errors:** Observed outcome = true function + random error.
 - **Errors are Independent:** Residuals (errors) are uncorrelated across observations.
 - **Homoskedasticity:** Error variance is constant across all levels of predictors.
 - **Normality of Errors (for inference):** Residuals approximately normally distributed (important for hypothesis testing, less for prediction).
-

5. Heuristic Search (*A*, Greedy Best-First)*

Definition:

Heuristic search algorithms use problem-specific knowledge (heuristics) to guide search more efficiently toward the goal, compared to blind search.

*Assumptions (for A optimality):**

- **Heuristic Admissibility:** Heuristic never overestimates the true remaining cost to the goal.
 - **Heuristic Consistency (Monotonicity):** Heuristic difference between nodes $\leq$ step cost. Ensures f-values (cost+heuristic) don't decrease along a path.
 - **Positive Step Costs:** All action costs are positive (no negative edges).
-

6. Uninformed Search (BFS, DFS, IDS)

Definition:

Uninformed search explores the search space without problem-specific heuristics, relying only on the structure of the state space.

Assumptions:

- **Finite Branching Factor:** Each state has a finite number of successors.
- **Finite Solution Depth:** The goal is reachable at some finite depth.
- **No Infinite Loops:** Either the state space is acyclic, or cycles are handled using a visited set.
- **Uniform Step Costs (for BFS optimality):** BFS guarantees optimal solution only if each step has equal cost.

1. Decision Tree Numerical

We have a training set of 12 cars with attributes **Colour** $\in \{\text{Red, Yellow, Green}\}$, **Type** $\in \{\text{Sports, SUV}\}$, **Origin** $\in \{\text{Domestic, Imported}\}$, and target **Stolen** $\in \{\text{Yes, No}\}$.

Training examples:

ID	Colour	Type	Origin	Stolen
1	Red	Sports	Domestic	Yes
2	Red	Sports	Domestic	Yes
3	Red	Sports	Imported	No
4	Yellow	Sports	Domestic	No
5	Yellow	Sports	Imported	Yes
6	Yellow	SUV	Imported	Yes
7	Yellow	SUV	Imported	No
8	Yellow	SUV	Domestic	No
9	Red	SUV	Imported	No
10	Red	SUV	Domestic	No
11	Green	Sports	Domestic	Yes
12	Green	SUV	Imported	Yes

Solution:

We will compute entropy and information gain exactly (log base 2). Important intermediate entropy values you'll see repeatedly:

- For a binary split with proportions 0.5 and 0.5: entropy = 1.0000
- For proportions 0.6 and 0.4 (or 0.4 and 0.6): entropy ≈ 0.97095
- For proportions 2/3 and 1/3 ($\approx 0.6667 / 0.3333$): entropy ≈ 0.918295
- For proportion 1 (pure): entropy = 0

(These are standard values:

$$H(0.5, 0.5) = 1.$$

$$H(0.6, 0.4) = -0.6 \log_2 0.6 - 0.4 \log_2 0.4 \approx 0.97095.$$

$$H(2/3, 1/3) \approx 0.918295.)$$

Step 1 — Overall entropy $H(S)$

Count classes in whole dataset:

- Yes: IDs {1,2,5,6,11,12} $\rightarrow$ 6 examples
- No : IDs {3,4,7,8,9,10} $\rightarrow$ 6 examples

So $p_{Yes} = 6/12 = 0.5$, $p_{No} = 0.5$.

Therefore overall entropy

$$H(S) = -0.5 \log_2 0.5 - 0.5 \log_2 0.5 = 1.0000.$$

Step 2 — Information gain for each attribute

We compute expected conditional entropy for each attribute and then information gain $IG = H(S) - H(S|\text{attribute})$.

A) Attribute Colour (values Red, Yellow, Green)

Partition counts:

- **Red** examples: IDs {1,2,3,9,10} → 5 examples: Yes = {1,2} → 2; No = 3.
→ $p_{Yes} = 2/5 = 0.4$, $p_{No} = 0.6$. Entropy $H(\text{Red}) \approx 0.97095$.
- **Yellow** examples: IDs {4,5,6,7,8} → 5 examples: Yes = {5,6} → 2; No = 3.
→ $H(\text{Yellow}) \approx 0.97095$.
- **Green** examples: IDs {11,12} → 2 examples: Yes = 2; No = 0.
→ $p_{Yes} = 1$. Entropy $H(\text{Green}) = 0$.

Weighted conditional entropy:

$$H(S|\text{Colour}) = \frac{5}{12} \cdot 0.97095 + \frac{5}{12} \cdot 0.97095 + \frac{2}{12} \cdot 0 = \frac{10}{12} \cdot 0.97095 = 0.809125.$$

So

$$IG(\text{Colour}) = 1.0000 - 0.809125 = 0.190875.$$

B) Attribute Type (Sports, SUV)

Partition:

- **Sports** examples: IDs {1,2,3,4,5,11} → 6 examples: Yes = {1,2,5,11} → 4; No = 2.
→ $p_{Yes} = 4/6 = 2/3$. Entropy $H(\text{Sports}) \approx 0.918295$.
- **SUV** examples: IDs {6,7,8,9,10,12} → 6 examples: Yes = {6,12} → 2; No = 4.
→ $p_{Yes} = 2/6 = 1/3$. Entropy $H(\text{SUV}) \approx 0.918295$.

Weighted conditional entropy:

$$H(S|\text{Type}) = \frac{6}{12} \cdot 0.918295 + \frac{6}{12} \cdot 0.918295 = 0.918295.$$

So

$$IG(\text{Type}) = 1.0000 - 0.918295 = 0.081705.$$

C) Attribute Origin (Domestic, Imported)

Partition:

- **Domestic**: IDs {1,2,4,8,10,11} → 6 examples: Yes = {1,2,11} → 3; No = 3. Entropy = 1.0000.
- **Imported**: IDs {3,5,6,7,9,12} → 6 examples: Yes = {5,6,12} → 3; No = 3. Entropy = 1.0000.

Weighted conditional entropy:

$$H(S|\text{Origin}) = \frac{6}{12} \cdot 1 + \frac{6}{12} \cdot 1 = 1.0000.$$

So

$$IG(\text{Origin}) = 1.0000 - 1.0000 = 0.0000.$$

Step 3 — Choose root attribute

Compare gains:

- $IG(\text{Colour}) \approx 0.190875$
- $IG(\text{Type}) \approx 0.081705$
- $IG(\text{Origin}) = 0.0000$

Best attribute = Colour. So root = Colour.

Step 4 — Split on Colour and continue growing tree

Branch Colour = Green

- Examples {11,12} → both **Yes** → pure leaf.
Leaf: Colour=Green ⇒ Stolen = Yes.

Branch Colour = Red

- Examples {1,2,3,9,10} → 5 examples: Yes=2, No=3. Entropy $H_{Red} \approx 0.97095$. Need to split further.

Compute gains restricted to Red subset for remaining attributes (Type, Origin).

Within Red:

- **Type** values in Red:
 - Sports: IDs {1,2,3} → Yes=2, No=1 → $p_{Yes} = 2/3 \Rightarrow \text{entropy} \approx 0.918295$.
 - SUV: IDs {9,10} → Yes=0, No=2 ⇒ entropy = 0.
Weighted: $H = (3/5) \cdot 0.918295 + (2/5) \cdot 0 = 0.550977$.
Gain within Red: $0.97095 - 0.550977 = 0.419973$.
- **Origin** values in Red:
 - Domestic: IDs {1,2,10} → Yes=2, No=1 ⇒ entropy ≈ 0.918295 .
 - Imported: IDs {3,9} → Yes=0, No=2 ⇒ entropy = 0.
Weighted: same as above = 0.550977. Gain = 0.419973.

Tie between Type and Origin. Choose **Type** (either is fine; we'll use Type).

So: Red → split on Type.

- **Red & Type = SUV:** IDs {9,10} → both No → pure leaf.
Leaf: Colour=Red & Type=SUV ⇒ Stolen = No.
- **Red & Type = Sports:** IDs {1,2,3} → Yes=2, No=1 → still mixed. Use Origin to split:
 - **Red & Sports & Origin = Domestic:** IDs {1,2} → both Yes → leaf Yes.
Leaf: Red & Sports & Domestic ⇒ Yes.
 - **Red & Sports & Origin = Imported:** ID {3} → No → leaf No.
Leaf: Red & Sports & Imported ⇒ No.

So Red branch completed.

Branch Colour = Yellow

- Examples {4,5,6,7,8} → 5 examples: Yes=2 (5,6), No=3 (4,7,8). Entropy ≈ 0.97095 .

Compute gains within Yellow:

- **Type** within Yellow:
 - Sports: IDs {4,5} → Yes=1, No=1 ⇒ entropy = 1.0
 - SUV: IDs {6,7,8} → Yes=1, No=2 ⇒ entropy ≈ 0.918295
Weighted $H = (2/5) \cdot 1 + (3/5) \cdot 0.918295 = 0.950977$.
Gain = $0.97095 - 0.950977 = 0.019973$.
- **Origin** within Yellow:
 - Domestic: IDs {4,8} → Yes=0, No=2 ⇒ entropy = 0.
 - Imported: IDs {5,6,7} → Yes=2, No=1 ⇒ entropy ≈ 0.918295
Weighted $H = (2/5) \cdot 0 + (3/5) \cdot 0.918295 = 0.550977$.
Gain = $0.97095 - 0.550977 = 0.419973$.

So **Origin** is best for Yellow. Split Yellow → Origin.

- **Yellow & Origin = Domestic:** IDs {4,8} → both No → pure leaf.
Leaf: Yellow & Domestic ⇒ Stolen = No.
- **Yellow & Origin = Imported:** IDs {5,6,7} → Yes=2 (5,6), No=1 (7) → mixed, entropy ≈ 0.918295 . We can split further by Type:

Within Yellow & Imported:

- **Type = Sports:** ID {5} → Yes → leaf Yes.
- **Type = SUV:** IDs {6,7} → Yes=1, No=1 → tie (entropy = 1). With no more attributes left to split (we've used Colour, Origin, Type), ID3 typically stops and uses majority class or can continue if there are other features. Here majority is ambiguous (1 vs 1) — ID3 can choose majority of parent or break ties arbitrarily. Common practice: break ties by majority of parent (Yellow&Imported parent had Yes=2, No=1 → majority Yes), or use a tie rule.

We'll use **majority at this node**: for Yellow & Imported & Type=SUV → predict **Yes** (because parent Yellow&Imported majority is Yes).

So:

- **Leaf: Yellow & Imported & Type=Sports ⇒ Yes** (pure)
- **Leaf: Yellow & Imported & Type=SUV ⇒ (tie) → predict Yes by majority of parent**

(Alternatively, if stricter purity is required, you could stop at Yellow&Imported and make that a leaf with majority class **Yes**.)

Final decision tree (compact)

- Root: **Colour**
 - **Green** → **Yes** (pure)
 - **Red** → split **Type**
 - **SUV** → **No** (pure)
 - **Sports** → split **Origin**
 - **Domestic** → **Yes**
 - **Imported** → **No**
 - **Yellow** → split **Origin**
 - **Domestic** → **No**
 - **Imported** → split **Type**
 - **Sports** → **Yes**
 - **SUV** → **(tie)** predict **Yes** by parent majority (or if using a different tie-break: could be No). We'll use **Yes**.

2. Linear Regression
- a. What is Linear Regression? Mention its assumptions

b. Solve this

Linear Regression Practice Question

Question:

A company wants to predict the **monthly electricity bill (in ₹)** for households using two features:

- x_1 = Number of rooms
- x_2 = Average monthly units consumed

The regression model is:

$$\hat{y} = w_0 + w_1x_1 + w_2x_2$$

Suppose the estimated parameters are:

- $w_0 = 500$
- $w_1 = 200$
- $w_2 = 15$

Training data (bills in ₹) for 3 households is given below:

Household	Rooms (x_1)	Units (x_2)	Actual Bill (y)
1	3	100	2500
2	4	120	3100
3	2	80	2200

Tasks:

1. Predict the bill for Household 2 using the regression model.
2. Compute the error (residual) for Household 2.
3. Using Mean Squared Error (MSE) cost function, compute the **gradient values** $\frac{\partial J}{\partial w_0}$, $\frac{\partial J}{\partial w_1}$, $\frac{\partial J}{\partial w_2}$ based on these 3 examples.



Solution:

Step 1: Prediction for Household 2

Model: $\hat{y} = 500 + 200x_1 + 15x_2$

For Household 2: $x_1 = 4, x_2 = 120$.

$$\hat{y} = 500 + (200)(4) + (15)(120) = 500 + 800 + 1800 = 3100$$

Predicted Bill = ₹3100

Step 2: Error (Residual) for Household 2

Residual = Predicted – Actual

$$e = 3100 - 3100 = 0$$

Residual = 0

Step 3: Gradient Computation

Cost function:

$$J(w) = \frac{1}{2m} \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)})^2$$

Gradients:

$$\frac{\partial J}{\partial w_j} = \frac{1}{m} \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)}) x_j^{(i)}$$

where $x_0 = 1$ for bias term.



(a) Compute predictions for all households

- Household 1: $500 + 200(3) + 15(100) = 500 + 600 + 1500 = 2600$.
- Household 2: already computed = 3100.
- Household 3: $500 + 200(2) + 15(80) = 500 + 400 + 1200 = 2100$.

$$\hat{y}_1 = 2600, \quad \hat{y}_2 = 3100, \quad \hat{y}_3 = 2100$$

(b) Compute residuals $e = \hat{y} - y$

- Household 1: $2600 - 2500 = 100$
- Household 2: $3100 - 3100 = 0$
- Household 3: $2100 - 2200 = -100$

Residuals: $[100, 0, -100]$

(c) Gradient values

- For w_0 (bias, $x_0 = 1$):

$$\frac{1}{3}(100 \cdot 1 + 0 \cdot 1 + -100 \cdot 1) = \frac{1}{3}(0) = 0$$

- For w_1 (Rooms):

$$\frac{1}{3}(100 \cdot 3 + 0 \cdot 4 + -100 \cdot 2) = \frac{1}{3}(300 + 0 - 200) = \frac{100}{3} \approx 33.3$$

- For w_2 (Units):

$$\frac{1}{3}(100 \cdot 100 + 0 \cdot 120 + -100 \cdot 80) = \frac{1}{3}(10000 + 0 - 8000) = \frac{2000}{3} \approx 666.7$$

c. How is it different from Logistic Regression?

d. Solve this:

Logistic Regression Practice Question

Question:

A logistic regression model is trained to predict whether a student **passes (y=1)** or **fails (y=0)** an exam based on two features:

- x_1 = Hours studied per week
- x_2 = Number of practice tests attempted

The model is:

$$P(y = 1|x) = \sigma(z), \quad z = w_0 + w_1x_1 + w_2x_2$$

where $\sigma(z) = \frac{1}{1 + e^{-z}}$.

The estimated parameters are:

- $w_0 = -6$
- $w_1 = 0.2$
- $w_2 = 1.0$

Tasks:

1. Predict the probability that a student who studies **20 hours** and takes **2 practice tests** passes the exam.
2. Decide the predicted class if the decision threshold is 0.5.
3. If the true label for this student was **1 (Pass)**, compute the contribution of this example to the **log-loss (cross-entropy loss)**.

Step 1: Compute linear score z

$$z = -6 + 0.2(20) + 1.0(2)$$

$$z = -6 + 4 + 2 = 0$$

Step 2: Compute probability with sigmoid

$$P(y = 1|x) = \sigma(0) = \frac{1}{1 + e^0} = 0.5$$

So, probability student passes = 0.5

Step 3: Predicted class (threshold = 0.5)

- If $P \geq 0.5 \rightarrow$ predict **Pass (1)**.
 - Here, $P = 0.5$ exactly, so prediction = **Pass (1)**.
-

Step 4: Log-loss (cross-entropy) contribution

Cross-entropy for one example:

$$\text{Loss} = -[y \log P + (1 - y) \log(1 - P)]$$

Here, $y = 1$, $P = 0.5$:

$$\text{Loss} = -[1 \cdot \log 0.5 + 0 \cdot \log(0.5)] = -\log 0.5$$

$$\text{Loss} = 0.693 \quad (\text{approx})$$

3. (i) Define Artificial Intelligence. Differentiate between strong AI and weak AI.
(ii) Explain the Turing Test with an example.
(iii) List four applications of AI in real life.
(iv) What are the key challenges of AI development?

Answer:

- (i) AI is the study of agents that perceive the environment and take actions to achieve goals. Strong AI $\rightarrow$ machines with human-like intelligence; Weak AI $\rightarrow$ task-specific systems.
- (ii) Turing Test checks if a machine can imitate human responses well enough to fool a human evaluator. Example: Chatbots.
- (iii) Examples: speech recognition, medical diagnosis, robotics, recommendation

systems.

(iv) Challenges: natural language understanding, reasoning under uncertainty, ethics, generalization.

4. (i) Define PEAS representation with an example of a self-driving car.
(ii) Differentiate between simple reflex agent, model-based agent, goal-based agent, and utility-based agent.
(iii) What are the properties of rational agents?
(iv) Explain deterministic vs stochastic environments with examples.

Answer:

(i) PEAS = Performance measure, Environment, Actuators, Sensors. For self-driving car: Performance = safety, fuel efficiency; Environment = roads, traffic; Actuators = steering, brakes; Sensors = cameras, LIDAR.

(ii) Reflex → rule-based, no memory; Model-based → keeps internal state; Goal-based → acts to achieve goals; Utility-based → maximizes overall performance.

(iii) Rational agent: selects actions to maximize expected performance measure.

(iv) Deterministic → next state fully determined (chess); Stochastic → has uncertainty (traffic navigation).

5. (i) Define BFS and DFS with examples.
(ii) Compare completeness, time complexity, and space complexity of BFS, DFS, IDS.
(iii) Explain why IDS is better than DFS.
(iv) Solve a given tree using BFS and DFS, show step order.

Answer:

(i) BFS explores level by level; DFS explores depth-first until dead-end. Example: maze solving.

(ii) BFS: Complete, $O(b^d)$ time, $O(b^d)$ space. DFS: Not complete (infinite depth), $O(b^m)$ time, $O(bm)$ space. IDS: Complete, optimal if unit cost.

(iii) IDS combines completeness of BFS with low memory of DFS.

(iv) Example solution → [expand nodes level-wise for BFS, depth-first for DFS].

6. (i) Define heuristic search with examples.
(ii) What are admissible and consistent heuristics?
(iii) Differentiate between greedy best-first search and A^* .
(iv) Solve the given search problem using A^* , showing $f(n) = g(n) + h(n)$.

Answer:

(i) Heuristic search uses problem-specific knowledge to guide search. Example: A^* uses shortest path + estimated distance.

(ii) Admissible = never overestimates true cost. Consistent = monotone across steps.

(iii) Greedy $\rightarrow$ uses $h(n)$ only, not optimal. $A^* \rightarrow$ uses $g(n)+h(n)$, optimal if h admissible.

(iv) Example $\rightarrow$ compute node expansions and f -values.

7. (i) Define conditional probability with example.

(ii) State and explain Bayes' theorem.

(iii) Differentiate between discrete and continuous probability.

(iv) A box contains 3 red, 2 blue balls. One is drawn. Find $P(\text{Red})$, $P(\text{Blue}|\text{not Red})$.

Answer:

(i) $P(A|B)$ = probability A occurs given B . Example: $P(\text{student passes} | \text{studied})$.

(ii) Bayes: $P(A|B) = P(B|A)P(A)/P(B)$.

(iii) Discrete = countable (coin toss), Continuous = ranges (height distribution).

(iv) $P(\text{Red})=3/5$, $P(\text{Blue}|\text{not Red})=2/2=1$.

8. (i) State Naïve Bayes classifier and its assumption.

(ii) Given a dataset with attributes "Word=Buy, Lottery" predict if email is Spam or Not Spam.

(iii) Why is Naïve Bayes still effective even if independence assumption is violated?

(iv) Explain Laplace smoothing with example.

Answer:

(i) NB applies Bayes theorem assuming conditional independence of features.

(ii) Work out posterior using counts $\rightarrow$ choose $\max P(\text{Class}|\text{Features})$.

(iii) Violations bias probabilities but relative rankings often still correct.

(iv) Laplace smoothing: add 1 to counts to avoid zero probability. Example: unseen word probability.

9. (i) Define bias and variance in ML models.

(ii) Explain training/validation/test split with example.

(iii) Define confusion matrix, precision, recall, F1-score.

(iv) How do learning curves help in diagnosing high bias vs high variance?

Answer:

(i) Bias = underfitting error, Variance = overfitting error.

(ii) Train = fit model, Validation = tune hyperparameters, Test = final evaluation.

Example: 60/20/20 split.

(iii) Confusion matrix = TP, TN, FP, FN. Precision = $TP/(TP+FP)$, Recall = $TP/(TP+FN)$, F1 = harmonic mean.

(iv) High bias $\rightarrow$ both curves high error. High variance $\rightarrow$ training low error, validation high error.

10. (i) Define the “curse of dimensionality” in the context of machine learning.
(ii) Explain how it affects:
 (a) Distance-based algorithms (e.g., k-NN, clustering).
 (b) Probabilistic classifiers like Naïve Bayes.
(iii) Show with a simple example how data sparsity increases with dimensions.
(iv) State two techniques to mitigate the curse of dimensionality.

(i) Definition:

The curse of dimensionality refers to the set of problems that arise when analyzing and organizing data in high-dimensional spaces that do not occur in low-dimensional settings. As the number of features (dimensions) grows, the volume of the space increases so fast that data becomes sparse, making statistical estimation and learning difficult.

(ii) Effects:

(a) On distance-based algorithms:

- In high dimensions, distances between points tend to become similar (relative contrast decreases).
- Nearest neighbor loses meaning since all points look equally far.
- Example: In 1D, nearest and farthest points differ greatly; in 100D, relative difference shrinks.

(b) On probabilistic classifiers (Naïve Bayes):

- Requires estimating conditional probabilities for each feature.
- With many features, the number of combinations explodes → probability estimates unreliable due to data sparsity.
- Leads to overfitting unless we assume independence or reduce dimensionality.

(iii) Example of sparsity:

Suppose we want to cover the unit interval $[0,1]$ with 10 equal bins. Each bin length = 0.1.

- In 1D → need 10 samples to have ~1 per bin.
- In 2D → $10 \times 10 = 100$ bins → need 100 samples.
- In 10D → 10^{10} bins → need 10 billion samples for same density.

Thus, higher dimensions require exponentially more data to maintain coverage.

(iv) Mitigation techniques:

- **Feature selection:** Keep only the most informative variables.
- **Dimensionality reduction:** Apply PCA, LDA, or autoencoders to project data into lower dimensions.
- **Regularization:** Penalize overly complex models to prevent overfitting.
- **Domain knowledge:** Engineer meaningful low-dimensional features.

11. Two points are drawn uniformly at random from a unit hypercube. Compare the ratio of minimum to maximum Euclidean distance between points as dimensionality increases.

Answer:

- In low dimensions, the minimum and maximum distances differ significantly.
- In very high dimensions, both distances approach the same value $\rightarrow$ ratio $\rightarrow 1$.
- This shows that in high dimensions, all points look equally far apart, which is a key manifestation of the curse of dimensionality.

12. Define the Naïve Bayes classifier in plain words.

List its key assumptions. For each assumption, briefly state what may happen if it is violated and one common remedy.

Explain Laplace (add-one) smoothing and why we use it.

1. Definition (plain):

Naïve Bayes is a simple probabilistic classifier that applies Bayes' theorem to compute the probability of each class given the observed features, and it assumes that features are conditionally independent given the class. The classifier picks the class with the highest posterior probability.

2. Key assumptions, effects if violated, and remedies:

- **Conditional independence (main Naïve assumption):**
 - *What it says:* Given the class, features are independent of each other.
 - *If violated:* Likelihoods are incorrectly factorized; probability estimates can be biased. However, classification accuracy often remains acceptable because relative scores may still separate classes.
 - *Remedy:* Remove or combine correlated features, use feature selection, use a model that captures dependencies (e.g., Bayesian networks, tree-augmented NB)

or switch to discriminative models (logistic regression).

- **Correct conditional distribution form for continuous features:**

- *What it says:* If using Gaussian NB, each continuous feature given class follows a Gaussian.
- *If violated:* Likelihoods are misestimated and probabilities become unreliable.
- *Remedy:* Transform features (log, sqrt), discretize, or use nonparametric density estimates.

- **Sufficient data / nonzero counts:**

- *What it says:* Need enough samples per feature–class combination.
- *If violated:* Zero counts lead to zero probabilities (which zero out posteriors).
- *Remedy:* Use smoothing (Laplace/add- α), collect more data, or merge rare categories.

- **Correct class priors (representative training data):**

- *What it says:* Priors estimated from training reflect test-time class frequencies.
- *If violated:* Posterior probabilities may be biased.
- *Remedy:* Adjust priors using domain knowledge or calibration.

3. Laplace (add-one) smoothing (plain):

When a categorical feature value never appears with a class in training, its estimated probability would be zero, and multiplying probabilities across features would make the whole posterior zero for that class.

Laplace smoothing adds a small pseudo-count (commonly +1) to each feature count, and increases the denominator accordingly, ensuring no zero probabilities. Formally for binary feature: $P(x = v \mid C) = \frac{\text{count}(x = v, C) + \alpha}{N_C + \alpha \cdot K}$, where N_C is count of class C, K is number of possible values (2 for binary), and α is smoothing ($\alpha=1$ for add-one).

Part B — Numerical (categorical features + Laplace smoothing)

You have the following training data for email classification (features are binary: Y = Yes, N = No):

ID	Offer	Urgent	Win	Class
1	Y	Y	Y	Spam
2	Y	N	Y	Spam
3	N	Y	N	NotSpam
4	Y	N	N	NotSpam
5	N	N	Y	Spam
6	N	N	N	NotSpam
7	Y	Y	N	Spam
8	N	Y	Y	Spam

(a) Compute class priors $P(\text{Spam})$ and $P(\text{NotSpam})$.

(b) Compute class-conditional probabilities for each feature value (use Laplace smoothing with $\alpha = 1$).

(c) For a new email with features Offer = N, Urgent = Y, Win = Y, compute the posterior probabilities and give the predicted class.

Question — Bias, Variance, Overfitting, Underfitting

Q. Bias, Variance, Overfitting, Underfitting

Part A — Theory (short answer)

1. Define bias and variance in the context of supervised learning.
2. Explain the **bias-variance tradeoff**.
3. Define overfitting and underfitting. Give one real-world example of each.
4. List two methods to reduce overfitting and two methods to reduce underfitting.

Part B — Numerical / Conceptual

A dataset is used to train two models for predicting house prices:

Model	Training Error	Validation Error	Complexity
M1	2.5	15.0	Low
M2	1.0	12.0	Medium
M3	0.5	20.0	High

1. Which model is likely **overfitting**? Which is **underfitting**? Justify using bias-variance reasoning.
2. If the goal is to minimize validation error, which model is preferable? Suggest a method to further improve generalization.
3. Draw a diagram showing training vs validation error vs model complexity, labeling regions corresponding to high bias / high variance.

Part C — Scenario (conceptual + practical)

Suppose you are building a linear regression model to predict exam scores from hours studied and sleep. You notice:

- Training error is low ($\approx 5\%$)
 - Validation error is very high ($\approx 25\%$)
1. What does this indicate about bias and variance?
 2. Suggest **3 strategies** to reduce the validation error while maintaining a low training error.

Part A — Theory

1. Bias and Variance

- **Bias:** Error due to oversimplified assumptions in the learning algorithm (underfitting). High bias → model cannot capture true patterns.
- **Variance:** Error due to sensitivity to small fluctuations in training data (overfitting). High variance → model performs well on training data but poorly on unseen data.

2. Bias-Variance Tradeoff

- Decreasing bias usually increases variance, and vice versa.
- Goal: Find a balance where total expected error ($\text{Bias}^2 + \text{Variance} + \text{Irreducible noise}$) is minimized.

3. Overfitting / Underfitting

- **Overfitting:** Model captures noise in training data → low training error, high test/validation error.
 - *Example:* 10th-degree polynomial for 8 points of data.
- **Underfitting:** Model too simple to capture underlying trend → high training and validation error.
 - *Example:* Linear regression on clearly quadratic data.

4. Remedies

- Reduce overfitting:
 - Use regularization (Ridge, Lasso)
 - Reduce model complexity (prune trees, lower polynomial degree)
 - Collect more training data
- Reduce underfitting:
 - Increase model complexity
 - Add more relevant features
 - Reduce regularization

Part B — Numerical / Conceptual

Given table:

Model	Training Error	Validation Error	Complexity
M1	2.5	15.0	Low
M2	1.0	12.0	Medium
M3	0.5	20.0	High

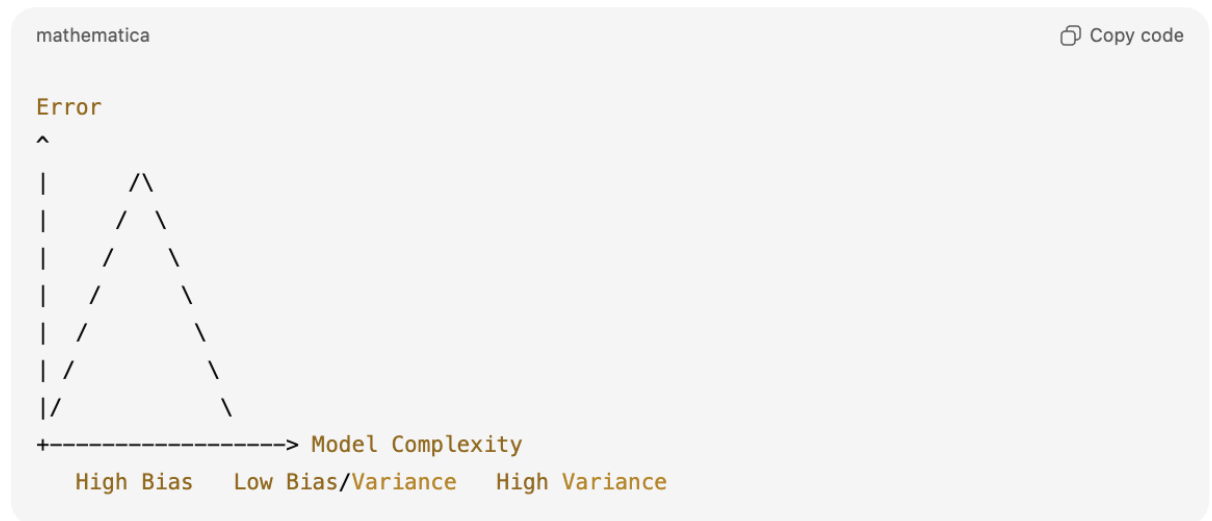
1. Overfitting / Underfitting

- **M1 (Low complexity):** High training error (2.5) + high validation error (15) → Underfitting (High bias, low variance).
- **M3 (High complexity):** Very low training error (0.5) + high validation error (20) → Overfitting (Low bias, high variance).
- **M2 (Medium complexity):** Training error 1.0, validation 12 → Moderate bias-variance tradeoff.

2. Preferred model to minimize validation error:

- **M2**, because validation error is lowest (12).
- Methods to improve generalization:
 - Regularization (L1/L2)
 - Cross-validation to tune hyperparameters
 - Ensemble methods (Bagging / Random Forest)

3. Diagram:



- Left side: Underfitting (high bias)
- Right side: Overfitting (high variance)
- Minimum total error somewhere in middle.

Part C — Scenario

1. Training error low ($\approx 5\%$) + validation error high ($\approx 25\%$)

- Bias is **low** (fits training data well)
- Variance is **high** (sensitive to training set $\rightarrow$ poor generalization)
- Conclusion: **Overfitting**

2. Strategies to reduce validation error:

- Add regularization (L1/L2)
- Increase training data or use data augmentation
- Simplify model (reduce number of features / use simpler basis functions)
- Cross-validation to select hyperparameters

Part B — Numerical (categorical + Laplace)

Given training data (repeated):

ID	Offer	Urgent	Win	Class
1	Y	Y	Y	Spam
2	Y	N	Y	Spam
3	N	Y	N	NotSpam
4	Y	N	N	NotSpam
5	N	N	Y	Spam
6	N	N	N	NotSpam
7	Y	Y	N	Spam
8	N	Y	Y	Spam

(a) Class priors

Count classes:

- Spam examples: IDs 1,2,5,7,8 → count = 5
 - NotSpam examples: IDs 3,4,6 → count = 3
- Total = 8

So:

- $P(\text{Spam}) = 5/8 = 0.625$
- $P(\text{NotSpam}) = 3/8 = 0.375$

(b) Class-conditional probabilities (with Laplace smoothing $\alpha = 1$)

We compute for each feature value $P(\text{feature} = v \mid \text{Class}) = \frac{\text{count} + 1}{N_{\text{class}} + 2}$ because binary $\rightarrow K=2$.

For Spam ($N_{\text{class}} = 5$):

- Offer = Y count among Spam? IDs 1(Y),2(Y),5(N),7(Y),8(N) $\rightarrow$ Y count = 3, N count = 2.
 $\rightarrow P(\text{Offer} = Y \mid \text{Spam}) = (3 + 1)/(5 + 2) = 4/7 \approx 0.5714$
 $\rightarrow P(\text{Offer} = N \mid \text{Spam}) = (2 + 1)/7 = 3/7 \approx 0.4286$
- Urgent = Y count among Spam? IDs 1(Y),2(N),5(N),7(Y),8(Y) $\rightarrow$ Y count = 3
 $\rightarrow P(\text{Urgent} = Y \mid \text{Spam}) = (3 + 1)/7 = 4/7 \approx 0.5714$
 $\rightarrow P(\text{Urgent} = N \mid \text{Spam}) = 3/7 \approx 0.4286$
- Win = Y count among Spam? IDs 1(Y),2(Y),5(Y),7(N),8(Y) $\rightarrow$ Y count = 4
 $\rightarrow P(\text{Win} = Y \mid \text{Spam}) = (4 + 1)/7 = 5/7 \approx 0.7143$
 $\rightarrow P(\text{Win} = N \mid \text{Spam}) = (1 + 1)/7 = 2/7 \approx 0.2857$

For NotSpam ($N_{\text{class}} = 3$):

- Offer = Y count among NotSpam? IDs 3(N),4(Y),6(N) $\rightarrow$ Y count = 1
 $\rightarrow P(\text{Offer} = Y \mid \text{NotSpam}) = (1 + 1)/(3 + 2) = 2/5 = 0.4$
 $\rightarrow P(\text{Offer} = N \mid \text{NotSpam}) = (2 + 1)/5 = 3/5 = 0.6$
- Urgent = Y count among NotSpam? IDs 3(Y),4(N),6(N) $\rightarrow$ Y count = 1
 $\rightarrow P(\text{Urgent} = Y \mid \text{NotSpam}) = 2/5 = 0.4$
 $\rightarrow P(\text{Urgent} = N \mid \text{NotSpam}) = 3/5 = 0.6$
- Win = Y count among NotSpam? IDs 3(N),4(N),6(N) $\rightarrow$ Y count = 0
 $\rightarrow$ With smoothing: $P(\text{Win} = Y \mid \text{NotSpam}) = (0 + 1)/5 = 1/5 = 0.2$
 $\rightarrow P(\text{Win} = N \mid \text{NotSpam}) = (3 + 1)/5 = 4/5 = 0.8$

Note: Without smoothing $P(\text{Win} = Y \mid \text{NotSpam})$ would be 0 and would zero out the whole posterior for NotSpam. Laplace smoothing prevents that.

(c) Posterior for new email Offer = N, Urgent = Y, Win = Y

Compute unnormalized posterior scores (class prior $\times$ product of likelihoods):

Score(Spam):

$$= P(\text{Spam}) \times P(\text{Offer} = N \mid \text{Spam}) \times P(\text{Urgent} = Y \mid \text{Spam}) \times P(\text{Win} = Y \mid \text{Spam})$$

Plug values:

- $P(\text{Spam}) = 0.625$
- $P(\text{Offer} = N \mid \text{Spam}) = 3/7$
- $P(\text{Urgent} = Y \mid \text{Spam}) = 4/7$
- $P(\text{Win} = Y \mid \text{Spam}) = 5/7$

So product $= 0.625 \times \frac{3}{7} \times \frac{4}{7} \times \frac{5}{7}$.

Compute numerator exactly as fraction first:

$$\frac{3 \cdot 4 \cdot 5}{7^3} = \frac{60}{343} \approx 0.175219.$$

Then multiply by 0.625:

$$\text{Score}(\text{Spam}) \approx 0.625 \times 0.175219 = 0.109512.$$

Score(NotSpam):

$$= P(\text{NotSpam}) \times P(\text{Offer} = N \mid \text{NotSpam}) \times P(\text{Urgent} = Y \mid \text{NotSpam}) \times P(\text{Win} = Y \mid \text{NotSpam})$$

Plug values:

- $P(\text{NotSpam}) = 0.375$
- $P(\text{Offer} = N \mid \text{NotSpam}) = 0.6$
- $P(\text{Urgent} = Y \mid \text{NotSpam}) = 0.4$
- $P(\text{Win} = Y \mid \text{NotSpam}) = 0.2$

Compute product:

$$0.375 \times 0.6 \times 0.4 \times 0.2 \downarrow 0.375 \times 0.048 = 0.018.$$

Plug values:

- $P(NotSpam) = 0.375$
- $P(Offer = N \mid NotSpam) = 0.6$
- $P(Urgent = Y \mid NotSpam) = 0.4$
- $P(Win = Y \mid NotSpam) = 0.2$

Compute product:

$$0.375 \times 0.6 \times 0.4 \times 0.2 = 0.375 \times 0.048 = 0.018.$$

Normalize to get posterior probabilities:

$$\text{Total} = 0.109512 + 0.018 = 0.127512$$

- $P(Spam \mid data) = 0.109512/0.127512 \approx 0.8587$
- $P(NotSpam \mid data) = 0.018/0.127512 \approx 0.1413$

Predicted class: Spam (posterior ≈ 0.859).

Comments / numerical-check:

- Without Laplace smoothing, $P(Win = Y \mid NotSpam) = 0$ would make $\text{Score}(NotSpam)=0$ and Spam posterior = 1. Smoothing gives a more realistic posterior and avoids degenerate zeroes.
- In practice compute log-probabilities (log priors + sum of log-likelihoods) to avoid underflow; compare log-scores.

Part C — Numerical (continuous feature → Gaussian Naïve Bayes)

Training examples for a binary class (Top vs NotTop) with one continuous feature `MinutesPlayed` :

- Top class: Minutes = [30, 35, 40]
- NotTop class: Minutes = [20, 25, 15]

Assume equal class priors. Use Gaussian Naïve Bayes where class-conditional density is $\mathcal{N}(\mu, \sigma^2)$ and estimate μ and σ^2 by the sample mean and MLE variance (divide by n). For a new instance with Minutes = 33:

- (a) Compute class means and variances.
- (b) Compute class-conditional densities $p(x = 33 \mid \text{class})$.
- (c) Compute posteriors and state predicted class.

Part C — Numerical (Gaussian Naïve Bayes)

Training data recalled:

- Top: Minutes = [30, 35, 40] $\rightarrow n_T = 3$
 - NotTop: Minutes = [20, 25, 15] $\rightarrow n_N = 3$
- Assume equal priors $P(\text{Top}) = P(\text{NotTop}) = 0.5$.

(a) Class means and MLE variances (divide by n)

Top:

- Mean $\mu_T = (30 + 35 + 40)/3 = 105/3 = 35$.
- Variance $\sigma_T^2 = \frac{(30-35)^2 + (35-35)^2 + (40-35)^2}{3} = \frac{25+0+25}{3} = 50/3 \approx 16.6667$.

NotTop:

- Mean $\mu_N = (20 + 25 + 15)/3 = 60/3 = 20$.
- Variance $\sigma_N^2 = \frac{(20-20)^2 + (25-20)^2 + (15-20)^2}{3} = \frac{0+25+25}{3} = 50/3 \approx 16.6667$.

(Here both classes happen to have same variance.)

(b) Class-conditional densities for $x = 33$

Gaussian PDF:

$$p(x \mid \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right).$$

Compute constants:

- $\sigma^2 = 50/3 \approx 16.6667$
 - $\sqrt{2\pi\sigma^2} = \sqrt{2\pi \times 16.6667} \approx \sqrt{104.7198} \approx 10.235$.
- So prefactor $\approx 1/10.235 \approx 0.0977$.

For Top ($\mu = 35$):

- $(x - \mu)^2 = (33 - 35)^2 = 4$.
- Exponent $= -4/(2\sigma^2) = -4/(33.3333) \approx -0.12$.
- $\exp(-0.12) \approx 0.8869$.
- So $p(33 \mid \text{Top}) \approx 0.0977 \times 0.8869 \approx 0.0867$.

For Top ($\mu = 35$):

- $(x - \mu)^2 = (33 - 35)^2 = 4$.
- Exponent $= -4/(2\sigma^2) = -4/(33.3333) \approx -0.12$.
- $\exp(-0.12) \approx 0.8869$.
- So $p(33 \mid Top) \approx 0.0977 \times 0.8869 \approx 0.0867$.

For NotTop ($\mu = 20$):

- $(x - \mu)^2 = (33 - 20)^2 = 169$.
- Exponent $= -169/33.3333 \approx -5.07$.
- $\exp(-5.07) \approx 0.0063$.
- So $p(33 \mid NotTop) \approx 0.0977 \times 0.0063 \approx 0.00062$.

(c) Posteriors and predicted class

Compute numerators:

- $\text{Num}(Top) = P(Top) p(33 \mid Top) = 0.5 \times 0.0867 \approx 0.04335$.
- $\text{Num}(NotTop) = 0.5 \times 0.00062 \approx 0.00031$.

Normalize:

- Total $\approx 0.04335 + 0.00031 = 0.04366$.
- $P(Top \mid x = 33) \approx 0.04335/0.04366 \approx 0.9929$.
- $P(NotTop \mid x = 33) \approx 0.0071$.

Predicted class: Top (very high posterior ≈ 0.993).

Practical note: always work in log-space for multiple continuous features to prevent underflow: $\log \text{posterior} \propto \log \text{prior} + \sum \log p(x_i \mid \text{class})$.

1. What is Informed Search? How does it differ from Uninformed Search?

Answer:

- **Informed Search (Heuristic Search):** A search strategy that uses **problem-specific knowledge (heuristics)** to guide the search towards the goal more efficiently.
- **Uninformed Search (Blind Search):** Has no extra information; it explores systematically without guidance.

Key Differences:

- *Uninformed*: Uses only the problem definition (start, actions, goal test). Example: BFS, DFS, UCS.
- *Informed*: Uses heuristic evaluation function $h(n)$ to estimate cost to reach the goal. Example: Greedy Best-First Search, A*.

Define heuristic. What are the properties of a good heuristic?

Answer:

- **Heuristic ($h(n)$)**: A function that estimates the cost of the cheapest path from node n to the goal.
- **Properties of a Good Heuristic**:
 1. **Admissibility**: Never overestimates the cost ($h(n) \leq h^*(n)$).
 2. **Consistency (Monotonicity)**: $h(n) \leq c(n, n') + h(n')$.
 3. **Efficiency**: Should be easy and fast to compute.
 4. **Accuracy**: Closer to the true cost improves search performance.

Explain Greedy Best-First Search. What are its advantages and disadvantages?

Answer:

- **Working**: Selects the node that appears closest to the goal using heuristic $h(n)$.
- **Advantages**:
 - Simple and fast.
 - Can find solutions quickly if heuristic is good.
- **Disadvantages**:
 - Not complete (may get stuck in loops).
 - Not optimal (may find suboptimal solutions).

4. What is the A Search algorithm? Why is it considered optimal?*

Answer:

- *A Algorithm:** Uses evaluation function
$$f(n) = g(n) + h(n)$$

where $g(n)$ = cost from start to n , and $h(n)$ = estimated cost from n to goal.
- **Optimality:**
 - If $h(n)$ is *admissible* and *consistent*, A* guarantees finding the **optimal path**.

5. Differentiate between Greedy Best-First Search and A Search.*

Answer:

Feature	Greedy Best-First	A* Search
Evaluation Function	$f(n) = h(n)$	$f(n) = g(n) + h(n)$
Focus	Goal proximity	Balance between cost so far & goal
Completeness	Not guaranteed	Guaranteed (if graph finite & h admissible)
Optimality	Not optimal	Optimal (with admissible heuristic)

6. What is the difference between admissible and consistent heuristics?

Answer:

- **Admissible heuristic:** Never overestimates the true cost to reach the goal. Ensures **optimality** in A*.
- **Consistent heuristic:** For every node n and successor n' ,

$$h(n) \leq c(n, n') + h(n')$$

Guarantees **monotonic $f(n)$** , making A* both optimal and efficient.

7. What is Iterative Deepening A (IDA) and why is it used?**

Answer:

- **IDA:** Combines depth-first search's low memory usage with A*'s heuristic guidance.
- Works by performing depth-first searches with increasing $f(n)$ cost thresholds.
- **Advantages:**
 - Uses memory proportional to depth ($O(d)$) unlike A* ($O(b^d)$).
 - Still guarantees optimality if heuristic is admissible.

8. What are real-world applications of Informed Search algorithms?

Answer:

1. **Pathfinding:** GPS navigation, robotics.
2. **Games:** Chess, Go, Pac-Man AI.
3. **Scheduling:** Job-shop scheduling, exam timetabling.
4. **Optimization Problems:** Resource allocation, network routing.
5. **Natural Language Processing:** Parsing, word-sense disambiguation.

Q12. Why is Iterative Deepening often preferred over BFS?

Answer:

- BFS requires huge memory $O(b^d)$.
- IDDFS requires only $O(bd)$ memory.
- IDDFS re-expands nodes but cost is small compared to memory savings.
- Hence, IDDFS is preferred when **state space is large and depth is unknown**.

Q13. Regularization

Regularization is a technique used to **prevent overfitting** by adding a **penalty term** to the loss function (or error function) that discourages overly complex models (e.g., very large parameter values).

Why Do We Need Regularization?

- When fitting a model (like polynomial curve fitting), if we only minimize the error on training data, we may get a very complex curve that **fits training data perfectly but performs badly on new data** — this is called **overfitting**.
- Regularization controls the **complexity** of the model, making it generalize better.

Mathematical Formulation

Suppose we are minimizing **Sum of Squares Error Function**:

$$E(w) = \frac{1}{2} \sum_{n=1}^N (y(x_n, w) - t_n)^2$$

where w = parameter vector, $y(x_n, w)$ = predicted output, t_n = target output.

With regularization, we add a penalty term:

$$E_{reg}(w) = \frac{1}{2} \sum_{n=1}^N (y(x_n, w) - t_n)^2 + \frac{\lambda}{2} \|w\|^2$$

where:

- λ = regularization parameter (controls strength of penalty)
- $\|w\|^2 = w_1^2 + w_2^2 + \dots + w_M^2$

This is called **L2 regularization** or **Ridge Regression**.

Intuition

- Without regularization $\rightarrow$ parameters w can become very large to fit every training point.
 - With regularization $\rightarrow$ large w are penalized, leading to smoother, simpler curves.
-

Types of Regularization

1. L2 Regularization (Ridge):

- Adds $\lambda \|w\|^2$
- Shrinks parameters smoothly towards zero.

2. L1 Regularization (Lasso):

- Adds $\lambda \|w\|_1 = \lambda(|w_1| + |w_2| + \dots)$
- Some parameters become exactly zero $\rightarrow$ performs **feature selection**.

(↴)

Effect of Regularization Parameter (λ)

- **Small λ** $\rightarrow$ very little penalty, may still overfit.
- **Large λ** $\rightarrow$ stronger penalty, may underfit (too simple model).
- We must choose λ carefully (e.g., using cross-validation).

★ Decision Trees: Structure, Entropy, Information Gain, Gini Impurity, Early Stopping, Tree Pruning, Advantages, Limitations, Use Cases

(Full 10-marks depth, all formulas copyable)

1. Introduction

A **Decision Tree** is a supervised learning algorithm used for both **classification** and **regression**. It recursively splits the dataset into subsets based on attribute values to create a tree-like structure.

2. Structure of a Decision Tree

A decision tree consists of:

- **Root Node** – the first split
- **Internal Nodes** – decision points on attributes
- **Branches** – outcome of decisions
- **Leaf Nodes** – final class label or predicted value

At each internal node, a feature is selected based on criteria like:

- Entropy / Information Gain
- Gini Impurity
- Variance reduction (for regression)

3. Entropy (Measure of Impurity)

Entropy quantifies disorder in a dataset.

Formula (copyable):

$$\text{Entropy}(S) = - \sum (p_i * \log_2(p_i))$$

Where:

p_i = probability of class i in dataset S .

Entropy ranges:

- $0 \rightarrow$ perfectly pure
- $1 \rightarrow$ maximally impure (for binary classes)

4. Information Gain (Feature Selection Metric)

Information Gain (IG) measures reduction in entropy after a split.

Formula (copyable):

$$\text{InformationGain}(S, A) = \text{Entropy}(S) - \sum (|S_v| / |S|) * \text{Entropy}(S_v)$$

Where:

- A = attribute
- S_v = subset after splitting on value v

- $|S_v|$ = number of samples in S_v

Goal: **maximize IG**.

5. Gini Impurity

Used in CART (Classification and Regression Trees).
Measures the probability of incorrect classification.

Formula (copyable):

$$\text{Gini}(S) = 1 - \sum (p_i^2)$$

Lower Gini represents a pure node.

6. Split Selection Using Gini

Formula (copyable):

$$\text{GiniSplit}(S, A) = \sum (|S_v| / |S|) * \text{Gini}(S_v)$$

Best feature = **minimum GiniSplit**

7. Stopping Criteria (Early Stopping)

Trees grow recursively until leaves are pure, but this overfits.
Early stopping halts tree growth based on:

- Minimum samples per leaf

- Minimum samples per split
- Maximum tree depth
- Information gain threshold
- Maximum number of nodes
- Maximum number of features per split

Goal: **reduce overfitting** and improve generalization.

8. Tree Pruning (Post-Pruning)

Pruning reduces size of fully-grown tree by removing weak branches.

Types:

8.1 Reduced Error Pruning

- Replace a subtree with a leaf if accuracy on validation set improves.

8.2 Cost Complexity Pruning (CART)

Penalized tree cost:

Formula (copyable):

$$R_{\alpha}(T) = R(T) + \alpha * |T|$$

Where:

- $R(T)$ = misclassification cost
- $|T|$ = number of leaves

- α = penalty parameter

Choose subtree T that minimizes $R_\alpha(T)$.

9. Decision Tree for Regression (Variance Reduction)

Entropy/Gini are for classification.
Regression trees use variance.

Node impurity (copyable):

$$\text{Var}(S) = (1 / |S|) * \sum (y_i - \text{mean}(S))^2$$

Variance reduction (copyable):

$$\text{VarianceReduction} = \text{Var}(S) - \sum (|S_v| / |S|) * \text{Var}(S_v)$$

10. Advantages of Decision Trees

1. Simple to understand and interpret
2. No need for feature scaling
3. Works for numerical and categorical data
4. Handles non-linear decision boundaries
5. No assumptions about data distribution
6. Fast to train and test

7. Can be used in ensemble methods (Random Forest, XGBoost)
-

11. Limitations

1. High risk of overfitting (trees grow too deep)
 2. Sensitive to small variations in data
 3. Greedy splitting may not find global optimum
 4. Biased toward features with many levels
 5. Instability → small changes cause different trees
 6. Can become large and complex without pruning
-

12. Use Cases

- Medical diagnosis
- Fraud detection
- Customer churn prediction
- Credit scoring
- Loan approval
- Image recognition (as part of ensembles)
- Marketing segmentation
- Game decision-making
- Failure prediction in machines

★ Introduction to Artificial Neural Networks (ANNs)

(Full 10-marks answer in text copyable format)

An **Artificial Neural Network (ANN)** is a computational model inspired by the structure and function of the biological brain. ANNs consist of interconnected processing units called **neurons**, which collectively learn patterns from data. ANNs are widely used in classification, regression, pattern recognition, natural language processing, vision, and control systems.

ANNs learn by adjusting the **weights** associated with connections between neurons based on an optimization rule (usually gradient descent).

✓ 1. ANN Structure

An ANN typically contains:

1. **Input Layer** – receives input features
2. **Hidden Layer(s)** – performs non-linear transformation
3. **Output Layer** – produces the final prediction

Each neuron computes a weighted sum of its inputs and applies an activation function.

✓ 2. Neuron Model (Mathematical Form)

For a neuron with inputs $x_1, x_2, \dots, x_n$ and weights $w_1, w_2, \dots, w_n$:

Net input (copyable):

$$\text{net} = \sum (w_i * x_i) + b$$

Output:

$$y = f(\text{net})$$

where

f = activation function

b = bias term

Common activation functions:

Sigmoid:

$$f(x) = 1 / (1 + e^{(-x)})$$

Step function:

$$f(x) = 1 \quad \text{if } x \geq 0$$

$$f(x) = 0 \quad \text{otherwise}$$

ReLU:

$$f(x) = \max(0, x)$$



Introduction to the Single-Layer Perceptron

A **Single-Layer Perceptron (SLP)** is the simplest form of neural network consisting of:

- One input layer

- One output neuron
- No hidden layers

The perceptron is a **binary linear classifier**.

✓ 1. Perceptron Output Function

Given:

$$\text{net} = \sum (w_i * x_i) + b$$

The perceptron uses a **threshold activation**:

Output (copyable):

$$y = 1 \quad \text{if } \text{net} \geq 0$$

$$y = 0 \quad \text{if } \text{net} < 0$$

Thus, a perceptron learns a **linear decision boundary** of the form:

Decision Boundary Equation:

$$\sum (w_i * x_i) + b = 0$$

This boundary separates the input space into two regions.

✓ 2. Perceptron Learning Rule

Weights are updated iteratively based on errors.

Error:

$$\text{error} = (\text{target} - \text{output})$$

Weight update rule (copyable):

$$w_i(\text{new}) = w_i(\text{old}) + \eta * \text{error} * x_i$$

Bias update:

$$b(\text{new}) = b(\text{old}) + \eta * \text{error}$$

where

η = learning rate

This rule ensures weights move toward reducing classification error.

Linear Separability

Linear separability is a fundamental concept for understanding the capability of perceptrons.

A dataset is **linearly separable** if there exists a linear boundary (a straight line in 2D, a hyperplane in n-D) that divides the classes perfectly.

Mathematical Condition (copyable):

There exists weights w and bias b such that:

$$w \cdot x + b \geq 0 \quad \text{for all samples in class 1}$$

$$w \cdot x + b < 0 \quad \text{for all samples in class 0}$$

If such a w and b exist $\rightarrow$ the dataset is linearly separable.

✓ Examples of Linear and Non-linear Separability

Linearly Separable Examples:

- AND gate
- OR gate

Their classes can be separated with a straight line.

Not Linearly Separable:

- XOR gate (requires multi-layer perceptrons)

A single-layer perceptron **cannot** learn XOR because no straight line exists that separates the classes.

★ Why Linear Separability Matters

A single-layer perceptron can ONLY solve problems that satisfy:

`class1 and class2 must be separable by a single linear hyperplane.`

If not, the model cannot converge, regardless of training duration.

✓ Geometric Interpretation

The perceptron computes:

$$w \cdot x + b = 0$$

This represents a **hyperplane**.

When input lies on one side → output is 1

Other side → output is 0

So the perceptron acts as a **linear classifier**.

★ Advantages of Single-Layer Perceptron

1. Simple and fast
 2. Easy to implement
 3. Works well on linearly separable tasks
 4. Computationally inexpensive
 5. Converges when solution exists (Perceptron Convergence Theorem)
-

★ Limitations

1. Cannot model non-linear decision boundaries
 2. Does not solve XOR or any non-linearly separable tasks
 3. Cannot approximate complex functions
 4. No hidden layers → limited representational power
-

★ Applications of Perceptrons

- Simple pattern recognition
- AND/OR logical classification
- Linearly separable dataset classification
- Early neural network foundational models
- Used as building block for deep neural networks

1. Multilayer Perceptron (MLP): Detailed Theory

A **Multilayer Perceptron (MLP)** is a type of **feed-forward neural network** consisting of multiple layers of neurons.

Each neuron performs a weighted sum of inputs followed by a nonlinear activation.

MLPs are the backbone of deep learning because they can approximate highly complex functions.

2. Architecture of an MLP

An MLP has:

1. **Input layer**
2. **One or more hidden layers**
3. **Output layer**

Each layer has a weight matrix and a bias vector.

Let:

- Input vector:
 $x = [x_1, x_2, \dots, x_n]$
 - Weight matrix of layer l :
 W_l
 - Bias vector of layer l :
 b_l
 - Activation of layer l :
 a_l
 - Pre-activation vector (weighted sum):
 z_l
-

3. Forward Propagation (exact formulas)

General formula for any layer l

$$z_l = W_l * a_{(l-1)} + b_l$$

$$a_l = \text{sigma}_l(z_l)$$

For the input:

$$a_0 = x$$

Where:

- $\text{sigma}_l()$ is the activation function (ReLU, sigmoid, tanh, etc.)
-

4. Example: Feedforward Computation in a 3-layer MLP

Suppose:

- Layer 1 = first hidden layer
- Layer 2 = second hidden layer
- Layer 3 = output layer

Then:

Layer 1

$$z_1 = W_1 * x + b_1$$

$$a_1 = \text{sigma}_1(z_1)$$

Layer 2

$$z_2 = W_2 * a_1 + b_2$$

$$a_2 = \text{sigma}_2(z_2)$$

Layer 3 (output)

$$z_3 = W_3 * a_2 + b_3$$

$$a_3 = \text{sigma}_3(z_3)$$

5. Why We Need Activation Functions

Without nonlinear activations, the network becomes a **linear** model:

$$W_2 * (W_1 * x) = (W_2 * W_1) * x$$

→ still linear, no matter how many layers.

Nonlinearity gives MLPs the power to learn complex functions.

6. Common Activation Functions (clean formulas)

Sigmoid

$$\text{sigma}(x) = 1 / (1 + \exp(-x))$$

Tanh

$$\tanh(x) = (\exp(x) - \exp(-x)) / (\exp(x) + \exp(-x))$$

ReLU

$$\text{ReLU}(x) = \max(0, x)$$

Softmax (for multi-class classification)

For class i:

$$\text{softmax}_i(z) = \exp(z_i) / \sum_j \exp(z_j)$$

7. Multiple Hidden Layers (Deep Networks)

Let there be H hidden layers.

For $l = 1, 2, \dots, H$:

$$z_l = W_l * a_{(l-1)} + b_l$$

$$a_l = \text{sigma}_l(z_l)$$

Output layer:

$$z_{(H+1)} = W_{(H+1)} * a_H + b_{(H+1)}$$

$$a_{(H+1)} = \text{sigma}_{(H+1)}(z_{(H+1)})$$

Interpretation

- Early layers learn **simple features**
(edges, frequency patterns, curves, local patterns)
- Deeper layers learn **abstract features**
(object parts, concepts, semantics)

This is called **hierarchical representation learning**.

8. Universal Approximation Theorem (Depth Insight)

A neural network with **one hidden layer** and a nonlinear activation can approximate **any continuous function**.

But:

- A deep network approximates with **fewer neurons**
- Deep networks learn **compositional structure** better

This is why multiple hidden layers are powerful.

9. Advantages of MLPs

1. Learn complex non-linear patterns

They can approximate any mathematical function.

2. End-to-end automatic feature extraction

No manual feature engineering.

3. General-purpose

Used in:

- Classification
 - Regression
 - NLP
 - Vision
 - Speech
 - Time-series modeling
-

10. Limitations

1. Large dataset requirement

They overfit on small data.

2. Computationally expensive

Many parameters → slow training.

3. Interpretability issues

Difficult to explain internal decisions.

4. Sensitive to hyperparameters

Learning rate, layer size, initialization, etc.

5. No built-in memory

Feedforward networks can't handle sequences (unlike RNNs).

11. When to Use MLPs

- When features are numeric
- When input is fixed-size vector
- When relationships are non-linear
- When hierarchical pattern extraction is beneficial

Examples:

- Predicting housing prices
- Fraud detection
- Classification of tabular data
- Function approximation in reinforcement learning

1. Forward Propagation (Deep Explanation)

=====

Forward propagation is the **process of computing outputs** of a neural network from the input.

Given:

- Input vector:
 $a_0 = x$
- For layer l :
Weights W_l , biases b_l , activation function σ_l .

Forward pass equations

Pre-activation (weighted sum):

$$z_l = W_l * a_{(l-1)} + b_l$$

Activation (output of layer):

$$a_l = \sigma_l(z_l)$$

The process repeats layer by layer until the final output a_L is computed.

Common activation functions

Sigmoid:

$$\sigma(x) = 1 / (1 + \exp(-x))$$

Derivative:

$$\sigma'(x) = \sigma(x) * (1 - \sigma(x))$$

Tanh:

$$\tanh(x) = (\exp(x) - \exp(-x)) / (\exp(x) + \exp(-x))$$

Derivative:

$$\tanh'(x) = 1 - \tanh(x)^2$$

ReLU:

$$\text{ReLU}(x) = \max(0, x)$$

Derivative:

$$\text{ReLU}'(x) = 1 \text{ if } x > 0 \text{ else } 0$$

2. Loss Function

After computing the output, we compute the loss.

Examples:

Mean Squared Error (MSE)

$$L = (1/2) * ||y - \hat{y}||^2$$

(where $\hat{y} = a_L$)

Cross-Entropy (binary)

$$L = -[y * \log(a_L) + (1 - y) * \log(1 - a_L)]$$

Cross-Entropy (multi-class softmax)

$$L = - \sum_i y_i * \log(a_{L_i})$$

Backpropagation computes $\partial L / \partial \mathbf{W}_l$ and $\partial L / \partial \mathbf{b}_l$.

=====

3. Backpropagation (Deep Theory)

=====

Backpropagation is reverse-mode differentiation.

It applies the chain rule layer-by-layer to compute gradients efficiently.

Key idea:

- Compute error at output
- Propagate backwards
- Compute gradients for every parameter

3.1 Error term (delta) for output layer

Let:

$$\delta_L = \partial L / \partial z_L$$

For softmax + cross entropy:

$$\delta_L = a_L - y$$

For sigmoid + cross entropy:

$$\delta_L = a_L - y$$

For MSE:

$$\delta_L = (a_L - y) * \sigma'_L(z_L)$$

3.2 Error term for hidden layers

For any hidden layer l :

$$\delta_l = (W_{(l+1)}^T * \delta_{(l+1)}) * \sigma'_l(z_l)$$

This is the **core backprop equation**.

=====

4. Gradients of Weights and Biases

Once δ_l is known:

Gradient of weights

$$\partial L / \partial W_l = \delta_l * a_{(l-1)}^T$$

Gradient of biases

$$\partial L / \partial b_l = \delta_l$$

Because bias is added to each neuron:

$$b_l \leftarrow b_l - \eta * \delta_l$$

5. Weight Update Rule

Using gradient descent:

Standard weight update

$$W_l \leftarrow W_l - \eta * (\partial L / \partial W_l)$$

$$b_l \leftarrow b_l - \eta * (\partial L / \partial b_l)$$

Where:

- η = learning rate (0.001, 0.01, etc.)

Substituting formulas

$$W_l \leftarrow W_l - \eta * (\delta_{l-1} * a_{(l-1)}^T)$$
$$b_l \leftarrow b_l - \eta * \delta_{l-1}$$

=====

6. Full Algorithm Summary (Copyable Pseudocode)

=====

Given: training sample (x, y)

1. Forward Propagation

$$a_0 = x$$

For $l = 1$ to L :

$$z_l = W_l * a_{(l-1)} + b_l$$
$$a_l = \text{sigma}_l(z_l)$$

2. Compute Loss $L(a_L, y)$

3. Backpropagation

$$\text{delta}_L = \partial L / \partial z_L$$

For $l = L-1$ down to 1:

$$\text{delta}_l = (W_{l+1}^T * \text{delta}_{l+1}) * \text{sigma}_l'(z_l)$$

4. Weight and Bias Update

For $l = 1$ to L :

$$W_l = W_l - \eta * (\text{delta}_l * a_{l-1}^T)$$

$$b_l = b_l - \eta * \text{delta}_l$$

7. Numerical Flow Example (Conceptual)

Suppose:

$$x = [0.5, 0.1]$$

$$y = [1, 0] \quad (\text{classification})$$

Layer 1:

$$z_1 = W_1 * x + b_1$$

$$a_1 = \text{sigma}(z_1)$$

Layer 2:

$$z_2 = W_2 * a_1 + b_2$$

$$a_2 = \text{softmax}(z_2)$$

Loss:

$$L = -\sum(y_i * \log(a_{2_i}))$$

Backprop:

$$\text{delta}_2 = a_2 - y$$

$$\text{delta}_1 = (W_2^T * \text{delta}_2) * \text{sigma}'(z_1)$$

Weight update:

$$W_2 = W_2 - \eta * (\text{delta}_2 * a_1^T)$$

$$W_1 = W_1 - \eta * (\text{delta}_1 * x^T)$$



8. Time Complexity

Forward pass

$O(\text{sum over layers } (n_l * n_{l-1}))$

Backward pass

Same order:

$O(\text{sum over layers } (n_l * n_{l-1}))$

So training a single sample is:

$O(\text{total number of weights})$

9. Summary for Exams

Forward propagation computes:

$z_l = W_l * a_{l-1} + b_l$

$a_l = \text{sigma}_l(z_l)$

Backpropagation computes:

$\text{delta}_L = \partial L / \partial z_L$

$\text{delta}_l = (W_{l+1}^T * \text{delta}_{l+1}) * \text{sigma}_l'(z_l)$

Gradients:

$$\partial L / \partial W_l = \text{delta}_l * a_{(l-1)}^T$$

$$\partial L / \partial b_l = \text{delta}_l$$

Weight update:

$$W_l \leftarrow W_l - \eta * (\text{delta}_l * a_{(l-1)}^T)$$

$$b_l \leftarrow b_l - \eta * \text{delta}_l$$

UNIT 4

1. Game Playing in AI — Full Depth Explanation

★ Introduction

Game playing has historically been one of the **core milestones** of AI research. It represents AI's ability to perform **strategic thinking**, **reason under competition**, **plan ahead**, and **optimize multi-step decisions**. Famous benchmarks include:

- 1997: IBM Deep Blue defeating Garry Kasparov
- 2016: DeepMind AlphaGo defeating Lee Sedol
- 2022: Meta Cicero (Diplomacy AI)

Games provide an ideal ground for developing **search**, **planning**, **heuristics**, and **learning**.

★ Formal Structure of Games in AI

A game is modeled mathematically as a **search problem**:

$\text{Game} = (S, A, T, s_0, \text{Terminal}, U)$

Where:

- S = set of states
- A = set of actions
- $T(s, a) \rightarrow s'$ = transition function
- s_0 = initial state
- Terminal = function returning True if game ends
- $U(s)$ = utility/payoff function of terminal states

Utility Function (copyable formula)

$\text{Utility}: S_{\text{terminal}} \rightarrow R$

Examples:

- Tic-tac-toe: +1 win, 0 draw, -1 loss
- Chess: engine evaluation score (centipawns)

★ Types of Games (Complete Classification)

1. Deterministic vs Non-deterministic

- Deterministic: actions fully determine next state

- Stochastic: includes probabilistic transitions
Example: Backgammon (dice)

2. Perfect vs Imperfect Information

Perfect information → full state visible (Chess)

Imperfect information → hidden information (Poker)

3. Cooperative vs Competitive

- Competitive: adversarial
- Cooperative: multi-agent shared reward

4. Zero-Sum vs Non-Zero-Sum

Zero-sum: $U_A + U_B = 0$

Non-zero-sum: both can win or lose independently

★ Game Representation: State-Space Model

Each state corresponds to a point in the game evolution.

Diagram:



★ Complexity Challenges

Game trees grow exponentially:

Basic Complexity Formula

Time Complexity = $O(b^d)$

Space Complexity = $O(bd)$

Where:

- b = branching factor
- d = depth of exploration

Examples:

- Tic-tac-toe: $9! \approx 362,880$ states
- Chess: $\approx 10^{120}$ states
- Go: $\approx 10^{360}$ states

Hence, heuristics and pruning are necessary.

★ Applications of Game Playing AI

- Game engines (Chess, Go, Poker, Diplomacy)
 - Security: attacker–defender models
 - Robotic adversarial planning
 - Economics: negotiation, auctions
 - Cybersecurity simulations
 - Multi-agent reinforcement learning
-

2. Two-Player Zero-Sum Games — Full Depth

★ Definition

A two-player zero-sum game is defined as:

$$\text{Utility_A}(s) + \text{Utility_B}(s) = 0$$

This models **pure competition**.

Formal Representation

A payoff function:

$$U(s) = +1 \text{ (win)}, 0 \text{ (draw)}, -1 \text{ (loss)}$$

For one player MAX:

$$\text{Utility_MIN} = -\text{Utility_MAX}$$

Properties

- Opponents are fully rational
- One player's gain = other's loss
- Strategies aim for optimal worst-case outcome

★ Game Model (Mathematical)

A zero-sum game is represented using a **normal form matrix**:

Example:

	B1	B2
A1	3	-2
A2	-1	4

MAX chooses rows, MIN chooses columns.

★ Minimax Value (copyable formula)

The minimax value of a state s is:

$$V(s) = \max(\min(U(\text{result}(s, a_{\min}))))$$

Formally:

$$V(s) = \begin{cases} \max_{\{a \in \text{Actions}(s)\}} V(\text{Result}(s, a)) & \text{if MAX turn} \\ \min_{\{a \in \text{Actions}(s)\}} V(\text{Result}(s, a)) & \text{if MIN turn} \end{cases}$$

★ Example: Tic-Tac-Toe

A solved zero-sum game:

- If both players play optimally $\rightarrow$ DRAW
- Minimax guarantees draw at worst for MAX

★ Applications

- Chess / Go engines
 - Security games
 - Multi-agent competition
 - Strategic conflict modeling
 - Bidding and Auctions
-

3. Game Tree Representation — Full Depth

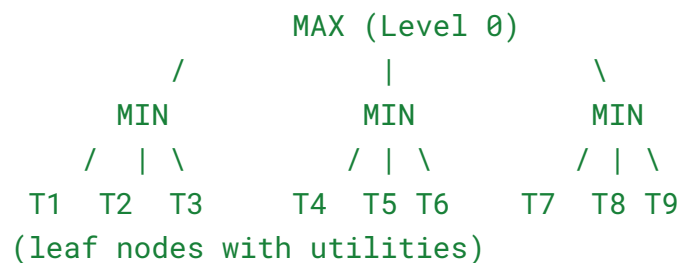
★ What is a Game Tree?

A **game tree** is a directed tree representing all possible sequences of moves from the start state.

Components

- Root = initial state
 - MAX/MIN alternating levels
 - Leaves = terminal states storing utility
-

★ Diagram (Copyable)



★ Branching Factor and Depth

- Branching factor (b): number of actions per state
- Depth (d): number of plies (half-moves)

Formula

Total Nodes = $O(b^d)$

Example:

- Chess: $b \approx 35$
 - Go: $b \approx 250$
-

★ Evaluation Function (Heuristic)

Used when reaching full depth is impossible.

General form:

$$\text{Eval}(s) = w_1 * f_1(s) + w_2 * f_2(s) + \dots + w_n * f_n(s)$$

Example for Chess:

- $f_1(s)$: material difference
 - $f_2(s)$: piece mobility
 - $f_3(s)$: king safety
-

★ Types of Game Trees

1. **Deterministic (Minimax)**
 2. **Stochastic (Expectimax)**
 3. **Partially Observable (Belief Trees)**
 4. **Monte Carlo Search Trees (MCTS)**
-

★ Use Cases

- Algorithms like Minimax, Alpha-Beta
 - Predicting sequences of moves
 - Strategy planning
-

4. Minimax Algorithm — Full Depth Explanation

★ Purpose

Minimax finds the **optimal adversarial action** assuming:

- MAX tries to maximize utility
 - MIN tries to minimize utility
-

★ Mathematical Definition

```
V(s) =  
    U(s)                                if terminal  
    max_{a ∈ A(s)} V(Result(s,a))      if MAX  
    min_{a ∈ A(s)} V(Result(s,a))      if MIN
```

★ Complete Pseudocode (Extended Version)

```
function MINIMAX(s, depth, player):  
    if TERMINAL(s) or depth == 0:  
        return EVALUATE(s)  
  
    if player == MAX:  
        best_value = -∞
```

```

    for a in ACTIONS(s):
        v = MINIMAX(RESULT(s,a), depth - 1, MIN)
        best_value = max(best_value, v)
    return best_value

else: # MIN player
    best_value = +∞
    for a in ACTIONS(s):
        v = MINIMAX(RESULT(s,a), depth - 1, MAX)
        best_value = min(best_value, v)
    return best_value

```

★ Example



Evaluation:

- MIN picks: 3, 2, 5
 - MAX picks max = 5
-

★ Complexity

Time = $O(b^d)$

Space = $O(b \cdot d)$

Minimax is optimal **only if**:

- search tree is complete

- utility is accurate
 - opponent plays perfectly
-

★ Limitations

- Exponential growth
 - Cannot be used in very-large games like Go
 - Needs pruning (Alpha-Beta)
 - Requires heuristics
-

5. Decision-Making in Adversarial Settings — Full Depth

★ Definition

Adversarial decision-making is the process of selecting actions while anticipating **hostile, competing, or strategic opponents**.

Examples:

- Chess
 - Cyber defence vs attacker
 - Financial trading adversaries
 - Autonomous vehicles avoiding hostile agents
-

★ Mathematical Model

Represented as a 2-agent adversarial environment:

State transition:

$$s_{(t+1)} = T(s_t, a_{MAX}, a_{MIN})$$

Utility:

$$U(s) = \text{Reward}_{MAX}(s) = -\text{Reward}_{MIN}(s)$$

Policies:

$$\pi_{MAX}: S \rightarrow A$$

$$\pi_{MIN}: S \rightarrow A$$

Value function (copyable formula):

$$V(s) = \max_{\{a_{max}\}} \min_{\{a_{min}\}} U(\text{result}(s, a_{max}, a_{min}))$$

★ Approaches to Adversarial Decision-Making

1. **Minimax** (perfect information)
 2. **Alpha-Beta pruning** (efficient minimax)
 3. **Expectimax** (stochastic games)
 4. **Opponent modeling** (non-optimal opponent)
 5. **Reinforcement learning (self-play)**
 6. **Monte Carlo Tree Search (MCTS)**
-

★ Components of a Decision-Making Model

- **Prediction:** modeling opponent's next action
 - **Evaluation:** scoring outcomes
 - **Optimization:** choosing best action
 - **Exploration vs exploitation**
 - **Uncertainty handling**
-

★ Example

Cybersecurity attacker–defender model:

- MAX = defender
 - MIN = attacker
 - Utilities represent compromise risk
-

★ Applications

- Military simulations
- Competitive markets
- Robotic soccer strategy
- Strategic negotiation systems
- Multi-agent reinforcement learning

6. ALPHA-BETA PRUNING (Ultra-Deep Explanation)

★ 6.1 Introduction

Alpha-beta pruning is an **optimization technique** for the **Minimax algorithm** used in adversarial game playing.

It **does not change the result**, only reduces the number of nodes explored.

The key idea:

“If a branch cannot influence the final decision, prune it.”

★ 6.2 Motivation (Why Do We Need Alpha-Beta?)

Game trees grow exponentially:

Time complexity of Minimax = $O(b^d)$

Where

- b = branching factor
- d = depth

For chess:

$b \approx 35$

$d \approx 80$

Nodes $\approx 35^{80} \gg$ number of atoms in universe

We cannot explore full game trees $\rightarrow$ need pruning.

★ 6.3 Alpha and Beta Values (Copyable Formula Definitions)

Alpha (α)

α = best value MAX can guarantee along the current path.

Beta (β)

β = best value MIN can guarantee along the current path.

Pruning Condition (Copyable)

If $\alpha \geq \beta \rightarrow$ prune further exploration.

Why?

MIN will not allow this branch to be selected, so MAX need not explore.

★ 6.4 Alpha-Beta Pseudocode (Highly Detailed)

```
function ALPHA_BETA(state, depth,  $\alpha$ ,  $\beta$ , player):
    if TERMINAL(state) or depth == 0:
        return EVALUATE(state)

    if player == MAX:
        value =  $-\infty$ 
        for action in ACTIONS(state):
            child = RESULT(state, action)
            value = max(value,
                        ALPHA_BETA(child, depth-1,  $\alpha$ ,  $\beta$ , MIN))
             $\alpha$  = max( $\alpha$ , value)
            if  $\alpha \geq \beta$ :
                break           //  $\beta$  cut-off (pruning)
```

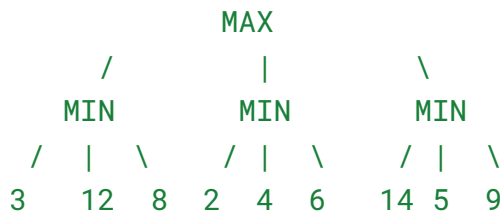
```

        return value

    else: // MIN player
        value = +∞
        for action in ACTIONS(state):
            child = RESULT(state, action)
            value = min(value,
                        ALPHA_BETA(child, depth-1, α, β, MAX))
            β = min(β, value)
            if α ≥ β:
                break // α cut-off (pruning)
        return value

```

★ 6.5 Example: Manual Pruning Diagram (ASCII)



While exploring the rightmost MIN subtree:

- Once MIN finds 5, and beta becomes $\leq$ alpha, the branch containing 9 is **pruned**.
-

★ 6.6 Complexity

Worst Case

$O(b^d)$ // same as minimax (no improvement)

Best Case (perfect ordering)

$O(b^{(d/2)})$ // reduces exponent by half $\Rightarrow$ exponential speedup

Interpretation

Searching to depth 10 takes the time of depth 5 minimax $\rightarrow$ **enormous win**.

★ 6.7 Benefits

- Makes deep search feasible
 - Enables real-time chess, Go, Checkers AIs
 - Allows using advanced heuristics and quiescence search
 - Critical for pruning bad move orders
-

★ 6.8 Use Cases

- Chess engines (Stockfish)
 - Checkers engines
 - Atari agents
 - Game-playing robotics
 - Real-time adversarial decision-making
-

7. PRUNING EFFICIENCY (Extreme Depth)

★ 7.1 Definition

Pruning efficiency measures **how much of the search tree is eliminated** by alpha-beta pruning.

Formula (copyable)

$$\text{Pruning Efficiency} = (\text{Nodes_without_pruning} - \text{Nodes_with_pruning}) / \text{Nodes_without_pruning}$$

The goal is to maximize pruning efficiency.

★ 7.2 Key Factors Affecting Pruning Efficiency

1. Move Ordering Quality

- Best moves first → Max pruning
- Worst moves first → Min pruning

Ordering heuristics:

- Killer heuristic
 - History heuristic
 - MVV-LVA (Move Most Valuable Victim - Least Valuable Attacker)
 - Captures first
 - Check moves first
-

2. Branching Factor Reduction

In chess:

$b \approx 35$ normally

α - β reduces effective $b \approx 6$

3. Depth of Search

Deeper search = more pruning opportunities

Iterative deepening improves pruning.

4. Heuristic Evaluations

Strong, consistent heuristics $\rightarrow$ better ordering $\rightarrow$ more pruning.

★ 7.3 Theoretical Optimal Efficiency

Perfect Ordering

$\text{Time} \approx O(b^{(d/2)})$

Reduction of exponent by half = **double depth search for same cost**.

★ 7.4 Practical Efficiency in Game Engines

Typical pruning:

60% – 95%

Stockfish pruning techniques:

- Alpha-beta
- Null move pruning

- Late move reductions
 - Futility pruning
 - Quiescence search
-

8. COMPLEX GAME STRATEGIES (Highly Detailed)

★ 8.1 What Are Complex Strategies?

Complex strategies are **multi-level, multi-step, long-horizon plans** that involve:

- Planning several moves ahead
 - Considering opponent counter-moves
 - Breaking the game into sub-goals
 - Using long sequences (tactics, end-games, openings)
 - Combining heuristics + search + learning
-

★ 8.2 Components of Complex Game Strategy

1. Opening Strategy

- Precomputed move sequences
- Known strong patterns
- Chess “Openings Book” or Go “Joseki”

2. Midgame Tactics

- Short combinations
- Threat creation
- Piece coordination
- Positional play

3. Endgame Strategy

- Simplified board
 - Known theoretical outcomes
 - Tablebase lookup (7-piece chess endgame database)
-

8.3 Strategic AI Techniques

1. Heuristic Evaluation Functions

Examples:

- Mobility
- Material advantage
- King safety
- Pawn structure

2. Pattern Databases

Used in:

- Rubik's cube

- 15-puzzle

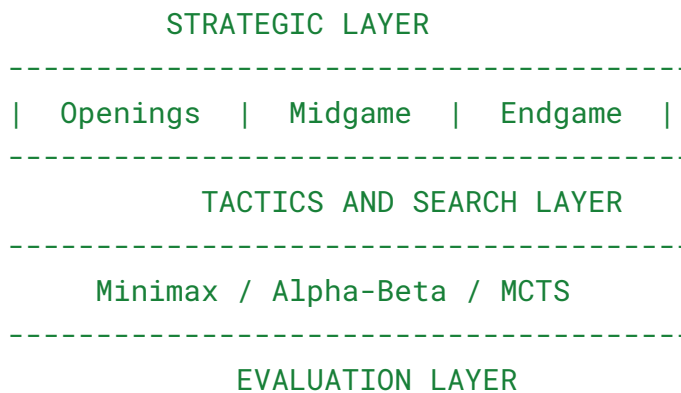
3. Monte Carlo Tree Search

Used in AlphaGo.

4. Reinforcement Learning with Self-Play

Used in AlphaZero.

★ 8.4 Game Strategy Diagram



9. EXPECTIMAX ALGORITHM (Ultra-Deep)

★ 9.1 Why Expectimax?

Minimax assumes **perfect opponent**.

But many games include **chance events**:

Examples:

- Dice (Ludo, Backgammon)
- Random card dealing (Poker)

- Random enemy movement (Pac-Man ghosts)

→ Need stochastic decision-making

→ Use **Expectimax**, not minimax.

★ 9.2 Types of Nodes

1. MAX nodes

AI chooses max expected return.

2. MIN nodes

Opponent chooses worst for AI.

3. CHANCE nodes

Used when outcomes are probabilistic.

★ 9.3 Expectimax Formula (Copyable)

Expectation Value

$\text{ExpectedValue}(s) = \sum [P(\text{outcome}_i) * \text{Value}(\text{outcome}_i)]$

Full Expectimax Definition

```
V(s) =  
    U(s)                                if terminal  
    max_{a} V(Result(s,a))              if MAX node  
    min_{a} V(Result(s,a))              if MIN node  
    Σ P(a) * V(Result(s,a))            if CHANCE node
```

★ 9.4 Pseudocode (Fully Detailed)

```
function EXPECTIMAX(state, depth, player):
    if TERMINAL(state) or depth == 0:
        return EVALUATE(state)

    if player == MAX:
        return max( EXPECTIMAX(child, depth-1, MIN)
                   for child in SUCCESSORS(state) )

    if player == MIN:
        return min( EXPECTIMAX(child, depth-1, MAX)
                   for child in SUCCESSORS(state) )

    if player == CHANCE:
        sum = 0
        for (child, p) in CHANCE_SUCCESSORS(state):
            sum += p * EXPECTIMAX(child, depth-1, MAX)
        return sum
```

★ 9.5 Applications

- Backgammon AI
 - Ludo, Snakes & Ladders, Monopoly
 - Gambling simulations
 - Pac-Man game AI
-

10. STOCHASTIC GAMES (Max Depth)

★ 10.1 Definition

A stochastic game is a game where **state transitions depend on both player actions and probabilities**.

★ 10.2 Formal Definition (Copyable)

A stochastic game is a tuple:

$$G = (S, A_1, A_2, P, R, \gamma)$$

Where:

- S = set of states
 - A_1, A_2 = actions of players
 - $P(s' | s, a_1, a_2)$ = transition probability
 - $R(s, a_1, a_2)$ = reward
 - γ = discount factor
-

★ 10.3 Types of Stochastic Games

1. **Turn-based stochastic games**
(Backgammon, Monopoly)
 2. **Simultaneous action stochastic games**
(Trading games, reinforcement learning)
 3. **Fully observable or partially observable games**
-

★ 10.4 Value Function for Stochastic Games

Copyable formula:

$$V(s) = \max_{a_1} \min_{a_2} [R(s, a_1, a_2) + \gamma * \sum P(s' | s, a_1, a_2) * V(s')]$$

This generalizes minimax + MDP.

★ 10.5 Examples

- Ludo
- Backgammon
- Dice-based board games
- RTS (Real-Time Strategy) games
- Stock market simulations
- Multi-agent reinforcement learning

1. Decision-Making Under Uncertainty

In many AI environments, an agent cannot perfectly predict the outcome of its actions. This leads to *uncertainty*, caused by:

- **Stochastic actions** (probabilistic outcomes)
- **Partial observability**
- **Sensor noise**
- **Unknown environment dynamics**

Hence, an intelligent agent must choose actions that **maximize expected utility** under uncertain outcomes.

Expected Utility Formula

$$EU(a | s) = \sum_{s'} P(s' | s, a) * U(s')$$

Where:

- $P(s'|s,a)$ = transition probability
 - $U(s')$ = utility of next state s'
-

★ 2. Markov Chains (Basics)

A **Markov Chain** is a probabilistic model describing a sequence of states where each state depends **only on the previous state**, not on the full history.

Markov Property

$$\begin{aligned} P(X_{t+1} = s' \mid X_t = s, X_{t-1}, \dots, X_0) \\ = P(X_{t+1} = s' \mid X_t = s) \end{aligned}$$

Components

1. **States** $S = \{s_1, s_2, \dots\}$

Transition Matrix T

$$T[i][j] = P(s_j \mid s_i)$$

- 2.
 3. **Initial State Distribution** π
-

Example Transition Matrix

	s1	s2	s3
s1	0.6	0.3	0.1
s2	0.2	0.5	0.3
s3	0.0	0.4	0.6

Diagram (ASCII)

```
[s1] --0.3--> [s2]
  |             |
  0.6           0.5
  v             v
[s3] <--0.4-- [s3]
```

Markov Chains are useful when:

- States are known
- Transition probabilities are known
- No actions are taken by the agent (pure dynamics)

★ 3. State Transitions

A **state transition** is written:

$$P(s' \mid s)$$

This tells the likelihood of moving to state **s'** from **s** in one step.

★ 4. Hidden Markov Models (HMMs)

An HMM extends Markov Chains to situations where:

- States are **hidden**

- Observations are **visible**

Definition

An HMM is a tuple:

$$\text{HMM} = (\mathbf{S}, \mathbf{O}, \mathbf{T}, \mathbf{Z}, \boldsymbol{\pi})$$

Where:

- $\mathbf{S}$ = hidden states
- $\mathbf{O}$ = observations
- $\mathbf{T}(\mathbf{s}, \mathbf{s}') =$ state transition probability
- $\mathbf{Z}(\mathbf{o} \mid \mathbf{s}) =$ observation probability
- $\boldsymbol{\pi}(\mathbf{s}) =$ initial distribution

HMM Diagram (ASCII)

```
Hidden:   s1 ----> s2 ----> s3 ---->
           T       T       T
```

```
Observed: o1       o2       o3
           ^       ^       ^
           Z       Z       Z
```

HMM Main Problems

(1) Filtering (Belief Update)

Estimate current hidden state given all past observations:

$$P(X_t \mid o_1, \dots, o_t)$$

(2) Prediction

Future state probabilities:

$$P(X_{t+k} \mid o_1, \dots, o_t)$$

(3) Smoothing

Past state estimation:

$$P(X_k \mid o_1, \dots, o_t)$$

(4) Viterbi Algorithm

Most likely hidden state sequence:

$$\operatorname{argmax} P(x_1, x_2, \dots, x_t \mid o_1 \dots o_t)$$

★ 5. Markov Decision Processes (MDPs)

MDPs model **decision-making under uncertainty** where the agent takes **actions**.

MDP Definition

$$\text{MDP} = (S, A, T, R, \gamma)$$

Where

- S = set of states
- A = set of actions
- $T(s, a, s') = P(s' \mid s, a)$ transition model
- $R(s, a, s') = \text{reward}$

- γ = discount factor ($0 \leq \gamma < 1$)
-

6. States, Actions, Transition Models, Rewards

States

Each represents a complete description of the environment.

Actions

Actions change states, but outcomes are stochastic.

Transition Model

$$T(s, a, s') = P(s' \mid s, a)$$

Reward Function

$R(s)$ (state reward)
 $R(s, a)$ (action reward)
 $R(s, a, s')$ (transition reward)

★ 7. Value Function $V(s)$

Measures how good a state is in the long run.

$V(s)$ = expected return starting from s

The return is:

$$\text{Return} = r_1 + \gamma r_2 + \gamma^2 r_3 + \dots$$

Thus:

$$V(s) = \sum P(s' \mid s, \pi(s)) * [R(s, \pi(s), s') + \gamma * V(s')]$$

★ 8. Action-Value Function $Q(s, a)$

Measures how good an action a is in state s .

$$Q(s, a) = \sum_{s'} P(s' | s, a) * [R(s, a, s') + \gamma * V(s')]$$

★ 9. Policies

A **policy** is a function:

$$\pi(s) = \text{action to take in state } s$$

Types of policies:

- **Deterministic:** $\pi(s) = a$
 - **Stochastic:** $\pi(a|s)$ = probability distribution
-

★ 10. Bellman Equations (Very Important)

10.1 Bellman Expectation Equation (for a given policy π)

State Value:

$$V_{\pi}(s) = \sum_a \pi(a|s) \sum_{s'} T(s, a, s') * [R(s, a, s') + \gamma * V_{\pi}(s')]$$

Action Value:

$$Q_{\pi}(s, a) = \sum_{s'} T(s, a, s') * [R(s, a, s') + \gamma * \sum_{a'} \pi(a' | s') Q_{\pi}(s', a')]$$

10.2 Bellman Optimality Equations

These define the **optimal value functions**.

Optimal State Value $V^*(s)$:

$$V^*(s) = \max_a \sum_{s'} T(s, a, s') * [R(s, a, s') + \gamma V^*(s')]$$

Optimal Action Value $Q^*(s, a)$:

$$Q^*(s, a) = \sum_{s'} T(s, a, s') * [R(s, a, s') + \gamma * \max_{a'} Q^*(s', a')]$$

★ 11. Optimal Policies

$$\pi^*(s) = \operatorname{argmax}_a Q^*(s, a)$$

A policy π^* is optimal if:

$$V_{\pi^*}(s) = V^*(s)$$

★ 12. Policy Evaluation

Given a policy π , compute $V_{\pi}(s)$.

Algorithm

1. Initialize $V(s) = 0$
2. Update using:

$$V(s) \leftarrow \sum_a \pi(a|s) \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V(s')]$$

3. Repeat until convergence.

★ 13. Iterative Value Calculation (Iterative Policy Evaluation)

Uses the above update until:

$$\max_s | V_{\text{new}}(s) - V_{\text{old}}(s) | < \theta$$

★ 14. Policy Iteration

Consists of:

1. Policy Evaluation

Compute $V_{\pi}(s)$

2. Policy Improvement

$$\pi_{\text{new}}(s) = \operatorname{argmax}_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V_{\pi}(s')]$$

Terminates when policy stops changing.

★ 15. Value Iteration

A simpler algorithm that merges policy evaluation + improvement.

Bellman Update

$$V(s) \leftarrow \max_a \sum_{s'} T(s,a,s') [R(s,a,s') + \gamma * V(s')]$$

Repeat until converged.

Extract policy:

$$\pi(s) = \operatorname{argmax}_a \sum_{s'} T(s,a,s') [R(s,a,s') + \gamma V(s')]$$

★ 16. Convergence

Both **Policy Iteration** and **Value Iteration** converge to:

- optimal value function V^*
- optimal policy π^*

Proof relies on contraction mapping (not required in exam).

★ 17. Applications of MDPs

1. Robot Navigation

- State: robot position
- Actions: move up/down/left/right
- Transition: movement may slip
- Reward: reaching goal or hitting obstacle

2. Resource Allocation

- Assigning limited resources under uncertainty

- CPU scheduling
- Inventory management

3. Autonomous Driving

- Lane changes
- Pedestrian prediction

4. Finance

- Optimal investment strategies
- Portfolio optimization

5. Healthcare

- Treatment planning
- Sequential medical decisions

6. Game Playing

- AI agents learning optimal moves

UNIT 5

TOPIC 1: Introduction to Reinforcement Learning (10-Marks Full Detailed Answer)

Reinforcement Learning (RL) is a learning paradigm in which an **agent learns to make sequential decisions** by interacting with an environment to **maximize cumulative rewards**.

Unlike supervised learning (where correct outputs are provided), RL learns through **trial-and-error** and **evaluating consequences of actions**.

★ 1.1 Core Idea of RL

At every time step t :

- The agent is in a **state**: s_t
- It chooses an **action**: a_t
- The environment gives a **reward**: r_t
- The environment moves to **new state**: s_{t+1}

The goal is to learn a **policy** π that **maximizes total discounted reward**:

Objective Function

$$G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots$$

Where:

- $\gamma \in [0,1]$ is **discount factor**
 - Higher $\gamma \rightarrow$ long-term reward importance
 - Lower $\gamma \rightarrow$ short-term reward importance
-

★ 1.2 RL Components (VERY IMPORTANT FOR EXAMS)

1. Agent

Learns through experience.

2. Environment

Responds to actions.

3. State (S)

Current situation.

4. Actions (A)

Choices available.

5. Reward (R)

Immediate feedback signal.

6. Policy (π)

Mapping from states to actions.

$\pi(a|s)$ = probability of taking action a in state s

7. Value Functions

Two major value functions:

State-value function:

$$V_{\pi}(s) = E_{\pi}[G_t \mid s_t = s]$$

Action-value function:

$$Q_{\pi}(s, a) = E_{\pi}[G_t \mid s_t = s, a_t = a]$$

1.3 Types of RL Algorithms

1. Model-based RL

Learns transition model $T(s, a, s')$ and reward model $R(s, a)$.

2. Model-free RL

Learns directly from experience:

- Temporal Difference (TD) Learning
- Q-Learning
- SARSA

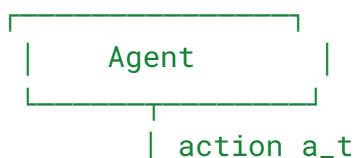
3. Policy Gradient / Actor-Critic

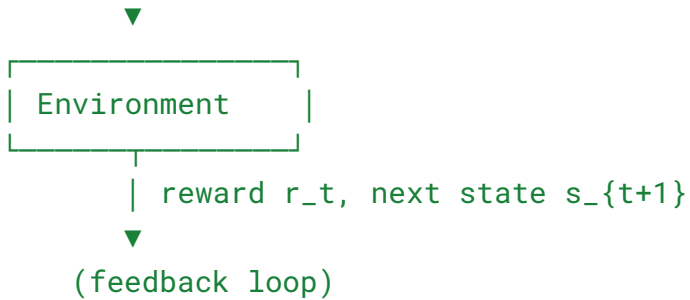
Directly improves policy.

★ 1.4 RL vs Supervised Learning vs Unsupervised Learning

Feature	RL	Supervised	Unsupervised
Objective	Max reward	Min loss	Find patterns
Feedback	Reward only	Correct label	No feedback
Data dependency	Interactive	Labeled	Unlabeled
Exploration	Required	Not needed	Not needed

★ 1.5 RL Process (Diagram)





★ 1.6 Example: Robot Navigation

Robot attempts to reach goal:

State	Action	Reward
Start	Move Forward	-1
Obstacle s	Move into obstacle	-5
Goal	Reach goal	+10

Through many episodes, robot learns:

- Avoid obstacles
 - Take shortest path
 - Maximize rewards
-

★ 1.7 Pseudocode (General RL Loop)

```
Initialize policy  $\pi$ 
for each episode:
    Initialize state s
    repeat until terminal:
```

Choose action a according to π
Take action a
Observe reward r and next state s'
Update value or Q-function
 $s = s'$

★ 1.8 Applications of RL

- Robotics (navigation & control)
 - Self-driving cars
 - Game AI (AlphaGo)
 - Finance & portfolio optimization
 - Recommendation systems
 - Industrial automation
 - Resource allocation (cloud computing)
-

★ 1.9 Advantages of Reinforcement Learning

- Learns without labeled data
 - Works in dynamic, uncertain environments
 - Learns optimal sequential decisions
 - Can handle huge state spaces (via deep RL)
-

★ 1.10 Limitations

- Requires a lot of interaction and time
- May converge to suboptimal policies
- Hard to train unstable deep RL models
- Exploration can be expensive/dangerous (robot crashing, etc.)

★ TOPIC 2: Q-Learning and Temporal Difference (TD) Learning (Full 10-Marks Detailed Answer)

(As requested, extremely detailed, exam-ready, including equations, example, diagrams, pseudocode, concepts, advantages.)

★ 1. Introduction to Q-Learning

Q-Learning is a fundamental **model-free Reinforcement Learning algorithm** used to learn the **optimal action-value function** without knowing the transition model or reward model of the environment.

It is based on **Temporal Difference (TD) learning** and is an **off-policy** method.

★ 2. Goal of Q-Learning

The goal is to learn the optimal **Q-function**:

$Q^*(s, a)$ = maximum expected future discounted return starting from state s , taking action a , and following the optimal policy thereafter.

Once $Q^*(s, a)$ is learned, the optimal policy is:

$$\pi^*(s) = \operatorname{argmax}_a Q^*(s, a)$$

★ 3. Q-Value and Bellman Optimality Equation

The Q-value update is based on the **Bellman Optimality equation**:

Bellman Optimality Equation

$$Q^*(s, a) = E[r + \gamma \max_{a'} Q^*(s', a')]$$

This expresses that the optimal Q-value equals the immediate reward plus the discounted value of the best possible action in the next state.

★ 4. Temporal Difference (TD) Learning

TD learning is used when:

- The environment is unknown
- The agent learns **from raw interaction**
- Learning happens after every step (not at episode end)

TD combines ideas of:

- **Monte Carlo** (learning from experience)
- **Dynamic Programming** (bootstrapping from other estimates)

TD Error (δ^t)

$$\delta_t = r + \gamma V(s_{t+1}) - V(s_t)$$

This error term is used to update value functions.

★ 5. Q-Learning Update Rule (KEY FORMULA)

The update rule for Q-Learning is:

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

Where:

- α = learning rate
- γ = discount factor
- $\max_{a'} Q(s', a')$ = best predicted value of next state
- r = reward

This is **off-policy** because the update uses **max(Q)**, not the action chosen by policy.

★ 6. Q-Learning Characteristics

Property	Q-Learning
Type	Model-free
Policy	Off-policy
Learns	Optimal Q-function
Uses	TD learning

Exploration Usually ϵ -greedy

★ 7. Example of One Q-Learning Update

State: $s = A$

Action: $a = \text{Right}$

Observed reward: $r = +2$

Next state: $s' = B$

Current $Q(B, *) = \{\text{Up} = 4, \text{Down} = 3\}$

Learning rate: $\alpha = 0.5$

Discount factor: $\gamma = 0.9$

Current $Q(A, \text{Right}) = 1$

Apply update:

$$Q(A, \text{Right}) \leftarrow 1 + 0.5 [2 + 0.9 * 4 - 1]$$

$$Q(A, \text{Right}) \leftarrow 1 + 0.5 [2 + 3.6 - 1]$$

$$Q(A, \text{Right}) \leftarrow 1 + 0.5 * 4.6 = 3.3$$

★ 8. Q-Learning Algorithm (Pseudocode)

Initialize $Q(s, a)$ arbitrarily

Repeat for each episode:

 Initialize state s

 Repeat for each step:

 Choose action a using ϵ -greedy policy from Q

 Take action a

 Observe reward r and next state s'

$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$

$s \leftarrow s'$

 until s is terminal

★ 9. Advantages of Q-Learning

- Can learn **optimal policy** even in stochastic environments.
 - Works without any environment model (model-free).
 - Converges to Q^* under mild conditions.
 - Simple to implement.
-

★ 10. Limitations

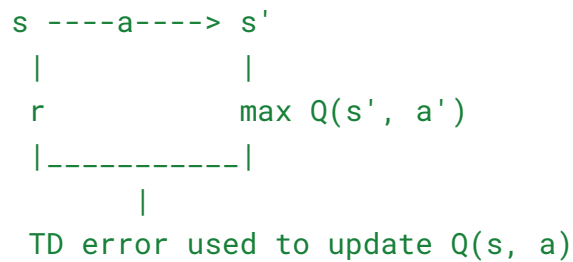
- Slow convergence for large state spaces.
 - Requires discretization if state space is continuous.
 - Exploration must be carefully balanced.
-

★ 11. Relationship Between TD and Q-Learning

Q-Learning is a **TD-control** method because:

- It updates value estimates using TD error.
 - It updates at every time step.
 - It bootstraps from other Q estimates.
-

★ 12. Diagram of Q-Learning Update



★ TOPIC 3: SARSA Algorithm (On-Policy Learning)

(Full 10-marks detailed explanation: theory, equations, example, diagrams, pseudocode, implications, exploration vs exploitation.)

★ 1. Introduction to SARSA

SARSA stands for:

State – Action – Reward – State – Action

It is a **model-free, on-policy, Temporal Difference (TD)** reinforcement learning algorithm.

Key Idea

SARSA learns the action-value function **while following the same policy that is being improved.**

This contrasts with Q-Learning, which is off-policy.

★ 2. Why SARSA is Called an On-Policy Method?

Because it updates Q-values using the **action actually taken** by the current policy.

Q-Learning uses $\max_{a'} Q(s', a')$

SARSA uses the action chosen *by the policy itself* at the next state.

Thus:

- SARSA learns what the agent **actually does**
 - Q-Learning learns what the agent **should ideally do**
-

★ 3. SARSA Update Rule (Very Important)

The SARSA TD update rule is:

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma Q(s', a') - Q(s, a)]$$

Where:

- s = current state
- a = action taken
- r = reward received
- s' = next state
- a' = next action chosen by the same policy
- α = learning rate
- γ = discount factor

This update follows the trajectory of the agent's policy.

★ 4. SARSA vs Q-Learning (Exam table)

Feature	SARSA	Q-Learning
Learning Type	On-policy	Off-policy
Update Uses	$Q(s', a')$ of chosen action	$\max Q(s', a')$
Behavior	Learns safe policies	Learns optimal policies
Exploration	Directly affects learning	Exploration not used in update
Good for	Risk-averse environments	Environments requiring optimality

★ 5. Example: Cliff Walking (VERY IMPORTANT Example)

A classical example demonstrating difference between SARSA and Q-Learning.

Grid world:

S _ _ _ _ _ _ _ _ G
 ■ ■ ■ ■ ■ ■ ■ ■ (cliff)

- Falling into cliff gives -100
- Moving normally gives -1

Behavior:

- **Q-Learning** finds optimal policy hugging the cliff edge (risky).

- **SARSA** learns safer path because its updates follow ϵ -greedy actions.

Thus, SARSA is **risk-aware**, Q-Learning is **risk-neutral**.

★ 6. SARSA Algorithm (Pseudocode — 10 marks importance)

```
Initialize  $Q(s,a)$  arbitrarily
For each episode:
    Initialize state  $s$ 
    Choose action  $a$  using  $\epsilon$ -greedy policy from  $Q$ 
    Repeat until terminal:
        Take action  $a$ 
        Observe reward  $r$  and next state  $s'$ 
        Choose next action  $a'$  using  $\epsilon$ -greedy policy from  $Q$ 
         $Q(s,a) \leftarrow Q(s,a) + \alpha [r + \gamma Q(s',a') - Q(s,a)]$ 
         $s \leftarrow s'$ 
         $a \leftarrow a'$ 
```

★ 7. Exploration vs. Exploitation Trade-off

SARSA explicitly uses the current policy for choosing the next action.

Common exploration policies:

ϵ -greedy

With probability $\epsilon \rightarrow$ explore (random action)

With probability $1-\epsilon \rightarrow$ exploit best action

Why exploration matters more in SARSA?

Because **the action chosen for update is the same action taken**.
Thus exploration directly influences learning.

In Q-Learning, exploration **does not affect the update** (off-policy).

★ 8. Discount Factor (γ) and Implications

The discount factor controls importance of future rewards:

$$0 \leq \gamma \leq 1$$

- γ close to 0 $\rightarrow$ agent becomes short-sighted
- γ close to 1 $\rightarrow$ agent considers long-term rewards

In SARSA:

A higher γ increases risk sensitivity because the policy used for update includes exploration.

★ 9. Convergence Property

SARSA converges to $\mathbf{Q}^\pi$, the Q-values of the ϵ -greedy policy.

If ϵ decreases over time, SARSA converges to **optimal $\mathbf{Q}^*$** .

★ 10. Applications of SARSA

- Autonomous driving (safe navigation)
- Robotics (risk-aware tasks)
- Resource management with uncertainty

- Grid-world navigation
- Flight control systems
- Financial RL where risk-awareness is essential

★ TOPIC 4: Exploration vs. Exploitation Trade-off

(Full 10-marks detailed explanation: deep theory, diagrams, formulas, examples, policies, implications)

★ 1. Introduction

In Reinforcement Learning (RL), an agent interacts with an environment to maximize cumulative reward.

To learn effectively, the agent must balance two competing goals:

Exploration

Trying **new actions** to discover potentially better rewards.

Exploitation

Choosing the **best-known action** based on past experience.

This creates the **exploration–exploitation dilemma**—one of the fundamental challenges of RL.

★ 2. Why Exploration is Necessary?

Without exploration:

- Agent may get stuck in **local optima**

- Never discovers better strategies
- Learns inaccurate value estimates

Example:

Agent always moves right because initially that gives a positive reward.
It never explores left, where a **much larger reward** might exist.

★ 3. Why Exploitation is Necessary?

Exploration alone leads to:

- Random wandering
- No use of previously learned knowledge
- Slow learning
- Low cumulative reward

Thus, exploitation is essential for **maximizing reward**.

★ 4. The Trade-off (Diagram)

EXPLORATION

- + Find new rewards
- + Learn true environment
- Risky, may reduce short-term reward

EXPLOITATION

- + Earn high reward now
- + Use existing knowledge
- May miss better actions

A good RL agent must **balance these two behaviors over time**.

★ 5. Mathematical View

Exploration–exploitation can be seen as optimizing:

Expected Return = Immediate Reward + Knowledge Gain

Where **knowledge gain** improves future rewards.

No closed-form solution → must use *exploration policies*.

★ 6. Exploration Policies (Very Important for Exams)

(1) ϵ -Greedy Policy

Most widely used.

With probability $(1 - \epsilon)$: choose best action (exploit)

With probability ϵ : choose random action (explore)

- When $\epsilon = 1$ → pure exploration
- When $\epsilon = 0$ → pure exploitation

Common choice:

$\epsilon = 0.1$

(2) Decaying ϵ -Greedy

To balance over time:

$\epsilon_t = \epsilon_0 / (1 + \text{decay_rate} * t)$

or exponential decay:

$$\epsilon_t = \epsilon_0 * \exp(-kt)$$

This ensures early exploration and late exploitation.

(3) Softmax / Boltzmann Exploration

Action probability:

$$P(a|s) = \exp(Q(s,a) / \tau) / \sum \exp(Q(s,b) / \tau)$$

Where τ is **temperature**:

- High $\tau \rightarrow$ random (exploration)
 - Low $\tau \rightarrow$ greedy (exploitation)
-

(4) Upper Confidence Bound (UCB)

Used in bandits & RL:

Choose a maximizing:

$$Q(s,a) + c * \sqrt{\ln(t) / N(s,a)}$$

Where:

- $N(s,a)$ = times action tried
- c = exploration factor

Encourages trying actions that are **uncertain**.

★ 7. Effect on RL Algorithms

SARSA

- On-policy → exploration affects learning
- Learns **safe** policies
- ϵ -greedy actions shape Q-values directly

Q-Learning

- Off-policy → exploration does NOT affect update
 - Learns optimal Q^*
 - More aggressive, risk-neutral
-

★ 8. Exploration in Continuous Spaces

Requires special methods like:

- Gaussian noise exploration
 - Ornstein–Uhlenbeck process (used in DDPG)
 - Entropy regularization (used in policy gradient methods)
-

★ 9. Practical Implications of Exploration

- Too much exploration → low reward
- Too little exploration → suboptimal policy

- Balanced approach → fastest convergence and best policy

Real-world systems must manage exploration **carefully**:

Example:

Robot exploring in real world → too much exploration = *damage or danger*.

★ 10. Example Demonstration

Suppose an agent is playing a simple game:

- Best known action: gives reward = 5
- Unknown action: untested, could give 10 or -10

If the agent always exploits, it gets 5 forever.

If it explores, it may discover 10, improving future reward.

★ 11. Exploration–Exploitation in Deep RL

Used in:

- DQN: uses ϵ -greedy
- Actor-Critic: uses entropy bonus
- PPO/A3C: stochastic policies encourage exploration

★ TOPIC 5: Discount Factor (γ) and Its Implications on Immediate vs Long-Term Rewards

(Full 10-marks detailed explanation: theory, math, diagrams, examples, implications, use cases)

★ 1. Introduction

In Reinforcement Learning (RL), an agent aims to maximize **cumulative future rewards**. However, rewards received **in the distant future** are usually considered **less valuable** than immediate rewards.

To model this mathematically, we use a **discount factor**, denoted by:

γ (gamma), where $0 \leq \gamma \leq 1$

The discount factor controls **how much future rewards matter**.

★ 2. What is the Discount Factor?

In RL, the total return from time t is:

$$G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \gamma^3 r_{t+3} + \dots$$

This is the **discounted sum of future rewards**.

Interpretation:

- $\gamma = 0 \rightarrow$ agent cares **ONLY immediate reward**
- $\gamma = 1 \rightarrow$ agent cares **equally about all future rewards**
- $0 < \gamma < 1 \rightarrow$ agent prefers **near-term** over distant rewards

★ 3. Why Do We Discount Rewards? (Theory)

(i) Uncertainty of future

Future states are unpredictable → discounting models uncertainty.

(ii) Mathematical stability

Discounting ensures the infinite reward sum **converges**.

(iii) Human/biological preference

Humans and many real systems value immediate rewards more.

(iv) Safety and responsiveness

Lower γ makes the agent react more to immediate outcomes.

★ 4. Bellman Equations and γ

State-value function:

$$V_{\pi}(s) = E_{\pi}[r + \gamma V_{\pi}(s')]$$

Action-value function:

$$Q_{\pi}(s, a) = E_{\pi}[r + \gamma Q_{\pi}(s', a')]$$

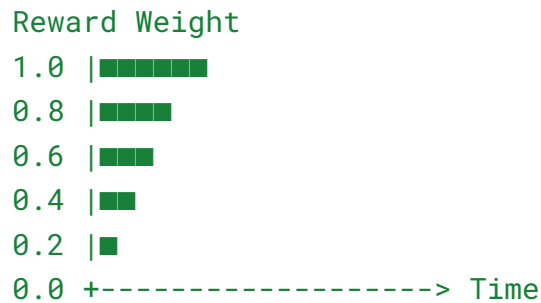
Optimal Bellman Equation:

$$V^*(s) = \max_a [R(s, a) + \gamma \sum T(s, a, s') V^*(s')]$$

The **discount factor controls** how much the agent considers the value of future states.

★ 5. Graphical Intuition

Reward discounting curve:



Future rewards shrink exponentially.

★ 6. Examples to Understand γ

Example 1: Short-sighted Agent ($\gamma = 0.2$)

Immediate reward = 5

Future reward in next state = 100

Value of future reward:

$$\gamma * 100 = 0.2 * 100 = 20$$

Agent chooses immediate 5 over future 20 → **short-term focus**.

Example 2: Long-term Agent ($\gamma = 0.95$)

$$\gamma * 100 = 0.95 * 100 = 95$$

Agent prefers to wait for future reward.

★ 7. Behavior Based on γ

$\gamma \rightarrow 0$ (myopic)

- Agent cares only about immediate reward
- Fast but unstable learning
- Good for reactive behaviors
- Example: avoiding collisions in robotics

$\gamma \rightarrow 1$ (far-sighted)

- Agent considers long-term rewards
- Learns more strategic behavior
- May explore more
- Example: strategic planning in chess, Go, long-term investment

★ 8. Mathematical Effects of γ

Total Return Convergence

For an infinite horizon:

$$G_t = \sum \gamma^k r_{t+k}$$

Converges iff $\gamma < 1$.

If $\gamma = 1$, return may diverge unless rewards are bounded.

★ 9. Interaction with Exploration

- High $\gamma \rightarrow$ encourages long-term gains $\rightarrow$ agent explores more
- Low $\gamma \rightarrow$ short-term immediate decisions $\rightarrow$ less exploration

Typically:

$0.90 \leq \gamma \leq 0.99$ for long-term tasks

$0.5 \leq \gamma \leq 0.8$ for short-term tasks

★ 10. Role of γ in Different RL Algorithms

SARSA

More sensitive to γ because on-policy policy includes exploration.

Q-Learning

Converges to optimal Q^* even with high γ .

Actor-Critic

γ affects critic's stability.

Value Iteration / Policy Iteration

γ directly influences Bellman updates and convergence speed.

★ 11. Use Cases

γ Value	Use Case
Low (0.0–0.5)	Immediate safety tasks (robot collisions)

Medium (0.5–0.8)	Inventory, resource allocation
High (0.9–0.99)	Chess, Go, long-term planning
≈ 1	Episodic tasks with delayed reward

★ Policy Iteration: Policy Evaluation and Policy Improvement (Full 10-Marks Answer with Copyable Formulas)

Policy Iteration is a fundamental Dynamic Programming (DP) algorithm used to compute the **optimal policy** for a Markov Decision Process (MDP). It alternates between:

1. **Policy Evaluation** – computing the value of the current policy
2. **Policy Improvement** – updating the policy using the computed values

The process repeats until the policy becomes **stable**.

1. MDP Components (Recap)

An MDP is defined by:

- **S**: set of states
- **A**: set of actions
- **P(s'|s,a)**: transition probability
- **R(s,a)**: reward
- **γ** : discount factor
- **$\pi(a|s)$** : policy

2. Objective

We want to find:

Optimal policy:

π^*

-

Optimal state-value function:

$V^*(s)$

-

Optimal action-value function:

$Q^*(s, a)$

-

Policy Iteration is guaranteed to converge to π^* .

3. Step 1: Policy Evaluation (Full Depth)

Given a fixed policy π , compute the value function:

★ **Bellman Expectation Equation (copyable)**

$$V\pi(s) = \sum_a \pi(a|s) \sum_{s'} P(s'|s, a) [R(s, a) + \gamma V\pi(s')]$$

This expresses $V\pi(s)$ in terms of future discounted returns.

★ Iterative Policy Evaluation Update (copyable)

$$V_{k+1}(s) = \sum_a \pi(a|s) \sum_{s'} P(s'|s,a) [R(s,a) + \gamma V_k(s')]$$

This update is repeated until convergence.

★ Q-value under policy π (copyable)

$$Q_{\pi}(s,a) = \sum_{s'} P(s'|s,a) [R(s,a) + \gamma V_{\pi}(s')]$$

4. Step 2: Policy Improvement

Once $V_{\pi}(s)$ is computed, the policy is updated using **greedy improvement**.

★ Greedy Policy Improvement Rule (copyable)

$$\pi_{\text{new}}(s) = \operatorname{argmax}_a \sum_{s'} P(s'|s,a) [R(s,a) + \gamma V_{\pi}(s')]$$

This chooses the action with the highest expected return.

★ Policy Stability Condition (copyable)

$$\pi_{\text{new}}(s) = \pi(s) \quad \text{for all states } s$$

If true $\rightarrow$ policy iteration stops $\rightarrow \pi$ is optimal.

5. Complete Policy Iteration Algorithm (copyable)

Initialize an arbitrary policy π

Repeat:

1. Policy Evaluation:

$$V\pi(s) = \sum_a \pi(a|s) \sum_{s'} P(s'|s,a) [R(s,a) + \gamma V\pi(s')]$$

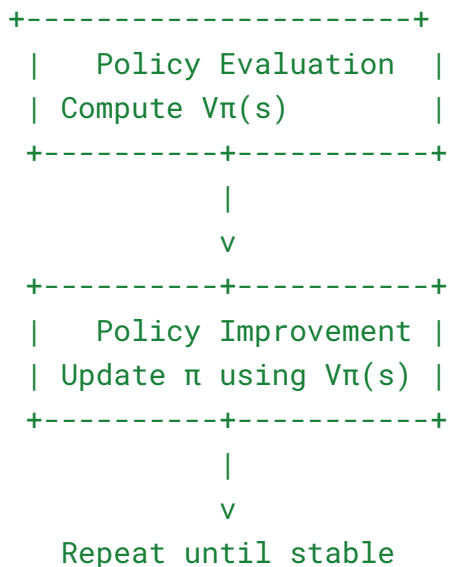
2. Policy Improvement:

$$\pi_{\text{new}}(s) = \operatorname{argmax}_a \sum_{s'} P(s'|s,a) [R(s,a) + \gamma V\pi(s')]$$

Until $\pi_{\text{new}} = \pi$

Return π

6. Diagram (ASCII)



7. Example (Illustrative)

Suppose initial policy:

$$\pi(s_1) = a_1$$

$$\pi(s_2) = a_2$$

After evaluating $V_\pi(s_1)$ and $V_\pi(s_2)$, we compute:

$$Q_\pi(s_1, a_1), Q_\pi(s_1, a_2) \\ Q_\pi(s_2, a_1), Q_\pi(s_2, a_2)$$

Then apply:

$$\pi_{\text{new}}(s) = \operatorname{argmax}_a Q_\pi(s, a)$$

If $\pi_{\text{new}} \neq \pi \rightarrow$ repeat evaluation.
If equal $\rightarrow \pi$ is optimal.

8. Time & Space Complexity

Policy Evaluation:

$$O(|S|^2 |A|)$$

-

Policy Improvement:

$$O(|S| |A|)$$

- - Converges in a small number of iterations.
-

9. Applications

- Robot navigation

- Optimal inventory control
- Traffic & elevator systems
- Resource allocation
- Gridworld planning

★ 1. VALUE ITERATION (10-MARKS FULL DEPTH ANSWER)

Value Iteration is a Dynamic Programming (DP) algorithm used in Markov Decision Processes (MDPs) to compute the **optimal value function** $V^*(s)$ and **optimal policy** π^* .

It differs from Policy Iteration by **combining policy evaluation and policy improvement into a single update step**.

1.1 Bellman Optimality Equation (Core of Value Iteration)

The Bellman optimality equation defines the value of a state under the optimal policy.

★ **Copyable Formula:**

$$V^*(s) = \max_a \sum_{s'} P(s' | s, a) * [R(s, a) + \gamma * V^*(s')]$$

Where:

- $V^*(s)$ = optimal state value
- a = action
- s' = next state

- $P(s' | s, a)$ = transition probability
- $R(s, a)$ = reward
- γ = discount factor

This equation is the foundation of Value Iteration.

1.2 Value Iteration Update Rule

Value Iteration uses the Bellman optimality equation iteratively.

★ **Copyable Formula:**

$$V_{k+1}(s) = \max_a \sum_{s'} P(s' | s, a) * [R(s, a) + \gamma * V_k(s')]$$

We start with arbitrary values V_0 , and repeatedly update.

This combines:

- **Policy evaluation**
- **Policy improvement**

... in a single equation.

1.3 Extracting the Optimal Policy

Once values converge:

★ **Copyable Formula:**

$$\pi^*(s) = \operatorname{argmax}_a \sum_{s'} P(s' | s, a) * [R(s, a) + \gamma * V^*(s')]$$

1.4 Value Iteration Algorithm (Copyable Pseudocode)

```
Initialize  $V(s)$  arbitrarily for all states  $s$ 
Repeat:
     $\Delta = 0$ 
    For each state  $s$ :
         $v = V(s)$ 
         $V(s) = \max_a \sum_{s'} P(s'|s,a) * [ R(s,a) + \gamma * V(s') ]$ 
         $\Delta = \max(\Delta, |v - V(s)|)$ 
Until  $\Delta < \theta$       # small threshold for convergence

 $\pi^*(s) = \operatorname{argmax}_a \sum_{s'} P(s'|s,a) [ R(s,a) + \gamma V(s') ]$ 
Return  $\pi^*$ 
```

1.5 Convergence Guarantee

Value Iteration converges because the Bellman operator is a **contraction mapping**, meaning:

★ **Copyable Formula:**

$$|V_{\{k+1\}}(s) - V^*(s)| \leq \gamma * |V_k(s) - V^*(s)|$$

So repeated Bellman updates converge exponentially fast.

1.6 Time and Space Complexity

Time complexity:

$$O(|S|^2 * |A|)$$

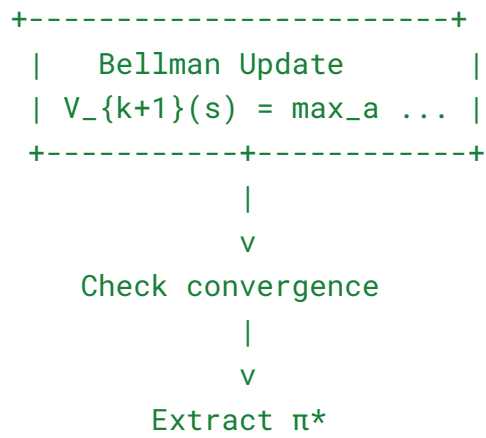
-

Space complexity:

$$O(|S|)$$

-
-

1.7 Diagram: Value Iteration



1.8 Applications of Value Iteration

- Robot navigation
- Autonomous driving
- Optimal routing
- Resource allocation
- Gridworld path planning

★ 2. TD LEARNING – ADVANTAGES OF TD(0) OVER MONTE CARLO (10-MARKS FULL DEPTH ANSWER)

Temporal Difference (TD) Learning and Monte Carlo (MC) methods are two major ways to learn value functions from experience.

2.1 TD(0) Update Rule

TD(0) updates using the **TD target**, which bootstraps from the next state.

★ **Copyable Formula:**

$$V(s_t) = V(s_t) + \alpha * [r_t + \gamma * V(s_{t+1}) - V(s_t)]$$

The term:

$$\delta_t = r_t + \gamma * V(s_{t+1}) - V(s_t)$$

is called **TD error**.

2.2 Monte Carlo Update Rule

Monte Carlo waits for the episode to finish and uses **actual return**:

★ **Copyable Formula:**

$$V(s_t) = V(s_t) + \alpha * [G_t - V(s_t)]$$

Where:

$$G_t = \sum_{k=0}^{T-t} \gamma^k * r_{t+k}$$

2.3 Advantages of TD(0) over Monte Carlo

✓ 1. TD(0) learns online, MC needs full episodes

- MC must wait until the episode ends.
- TD(0) updates after every step → more efficient.

✓ 2. TD(0) works for continuing (non-terminating) tasks

MC cannot be used when episodes never end.
TD(0) works perfectly.

✓ 3. TD(0) has lower variance

MC target:

$G_t = \text{high variance}$

TD(0) target:

$r_t + \gamma * V(s_{t+1}) = \text{low variance}$

Bootstrapping reduces noise.

✓ 4. TD(0) converges faster

- MC needs more data per update.
- TD(0) learns from partially completed sequences.

✓ 5. TD(0) is more memory efficient

No need to store whole episode.

✓ 6. TD(0) forms the basis of advanced RL algorithms

Like Q-learning, SARSA, Actor–Critic etc.

MC is less used in modern RL.

2.4 Summary Table

Feature	TD(0)	Monte Carlo
Needs full episode?	No	Yes
Works for non-terminating tasks?	Yes	No
Variance	Low	High
Bias	Higher	Zero
Learning speed	Fast	Slow
Memory usage	Low	High

★ Introduction to Actor–Critic Methods: Architecture and Advantages (Full 10-Marks Answer)

Actor–Critic methods are a powerful class of Reinforcement Learning algorithms that **combine the strengths of policy-based and value-based methods**. They simultaneously learn:

1. **An Actor** — the policy
2. **A Critic** — the value function

This leads to faster learning, lower variance, and better stability than using policy gradient or value iteration alone.

1. ✓ Fundamental Idea

Policy-based methods (like REINFORCE) optimize the policy parameters directly but suffer from:

- **High variance**
- **Slow convergence**
- **No value function** to estimate the quality of actions

Value-based methods (like Q-learning):

- Cannot model continuous actions
- Indirectly obtain the policy
- May be unstable

Actor–Critic combines both.

Actor: learns the policy

Critic: evaluates how good the chosen action was

2. ✓ Actor–Critic Architecture (Full Description)

2.1 Actor (Policy Network)

The Actor represents the policy:

$$\pi_{\theta}(a \mid s)$$

where

θ = parameters of the Actor network

The Actor chooses an action based on the state.
It is updated using feedback from the Critic.

2.2 Critic (Value Network)

The Critic estimates the value function:

State-value:

$$V_w(s)$$

Action-value:

$$Q_w(s, a)$$

where

w = parameters of the Critic network

The Critic provides a **baseline** to reduce the variance of policy gradients.

2.3 Advantage Function

The Critic helps compute the **advantage**:

$$A(s, a) = Q(s, a) - V(s)$$

This tells the Actor how much better an action was compared to the average.

Often approximated using TD-error:

$$A(s, a) \approx \delta$$

3. ✓ Temporal Difference (TD) Error

TD-error is the core learning signal in Actor–Critic:

$$\delta = r + \gamma * V(s') - V(s)$$

where

r = reward

γ = discount factor

s' = next state

This error is used to:

- Train the Critic (value prediction)
 - Improve the Actor (policy update)
-

4. ✓ Parameter Updates

4.1 Critic Update

Critic minimizes TD-error:

$$w \leftarrow w + \beta * \delta * \nabla_w V_w(s)$$

β = learning rate for the Critic

4.2 Actor Update (Policy Gradient Using TD Error)

$$\theta \leftarrow \theta + \alpha * \delta * \nabla_{\theta} \log \pi_{\theta}(a | s)$$

α = learning rate for the Actor

This formula means:

- If $\delta > 0 \rightarrow$ action was good $\rightarrow$ increase probability

- If $\delta < 0 \rightarrow$ action was bad $\rightarrow$ decrease probability
-

5. ✓ Complete Actor–Critic Algorithm (Text Copyable)

Initialize parameters θ (actor), w (critic)

Loop for each episode:

 Initialize state s

 Loop for each step:

 Choose action $a \sim \pi_{\theta}(a | s)$

 Execute a and observe reward r and next state s'

 Compute TD error:

$$\delta = r + \gamma * V_w(s') - V_w(s)$$

 Critic update:

$$w \leftarrow w + \beta * \delta * \nabla_w V_w(s)$$

 Actor update:

$$\theta \leftarrow \theta + \alpha * \delta * \nabla_{\theta} \log \pi_{\theta}(a | s)$$

$s \leftarrow s'$

6. ✓ Advantages of Actor–Critic Methods (Very Deep)

1. Lower Variance in Policy Gradient Updates

Since the Critic provides a baseline $V(s)$, the gradient variance drops dramatically.

2. Works in Continuous Action Spaces

Because the Actor outputs *parameters of a distribution*, it supports continuous control.

Example:

$$\pi_{\theta}(a \mid s) = \text{Normal}(\mu_{\theta}(s), \sigma_{\theta}(s))$$

3. More Sample Efficient

Critic reuses experience to estimate values → faster learning.

4. Combines Strengths of Two Worlds

- Actor → optimal policy learning
- Critic → value estimation & stability

5. Supports On-policy and Off-policy variants

- A2C / A3C (on-policy)
- DDPG, TD3, SAC (off-policy, deep RL)

6. Reduces Instability

Pure policy gradient fluctuates; Critic anchors learning using TD updates.

7. Better Scalability to Large State Spaces

Neural networks easily model:

- $V(s)$
- $\pi(a|s)$
- $Q(s, a)$

7. ✓ Why Actor–Critic is Better Than Pure Policy Gradient

Pure Policy Gradient	Actor–Critic
High variance	Low variance
No value function	Learns $V(s)$
Slow convergence	Faster convergence
Does not generalize well	Stable generalization
Requires full trajectory return G	Uses TD-error δ

8. ✓ Where Actor–Critic is Used (Real Applications)

- Robotics (locomotion, manipulation)
- Self-driving vehicles
- Trading and portfolio control
- Game-playing (AlphaGo uses a form of Actor–Critic)
- Industrial control systems
- Recommendation systems
- Navigation tasks (continuous control)

PRACTICE QUESTIONS on end sem topics only

Q1. (Theory + Numerical)

Explain linear separability in perceptrons. Using a numerical example, check whether the dataset is linearly separable and find perceptron weights using the perceptron learning rule (one epoch only).

Answer

Theory: Linear Separability

A dataset is **linearly separable** if a single straight line (in 2D), plane (in 3D), or hyperplane (in n-D) can separate all positive and negative class points.

A perceptron converges **only when data is linearly separable**.

The decision boundary is:

$$w_1 * x_1 + w_2 * x_2 + b = 0$$

Numerical Example

Dataset:

x1	x2	Target (t)
0	0	0
0	1	0
1	0	0

1 1 1

This is the **AND gate**, which is linearly separable.

Initialize Weights

```
w1 = 0
w2 = 0
b = 0
learning_rate = 1
```

Perceptron Update Rule

Activation:

```
y = 1 if (w1*x1 + w2*x2 + b) >= 0 else 0
```

Weight update:

```
w1 = w1 + η*(t - y)*x1
w2 = w2 + η*(t - y)*x2
b = b + η*(t - y)
```

Epoch 1 Calculations

1. $x = (0,0)$, $t=0$

```
y = 1 (because 0 >= 0)
t - y = -1
w1 = 0
w2 = 0
b = -1
```

2. $x = (0,1)$, $t=0$

$y = 0$ (because $w_1 \cdot x_1 + w_2 \cdot x_2 + b = -1 < 0$)
 $t - y = 0$ (no update)

3. $x = (1,0)$, $t=0$

$y = 0$
 $t - y = 0$

4. $x = (1,1)$, $t=1$

$net = w_1 \cdot 1 + w_2 \cdot 1 + b = 0 + 0 - 1 = -1$
 $y = 0$
 $t - y = 1$

$w_1 = 1$
 $w_2 = 1$
 $b = 0$

Final weights after 1 epoch:

$w_1 = 1$
 $w_2 = 1$
 $b = 0$

Q2. (Theory + Numerical)

Explain the architecture of a Multi-Layer Perceptron (MLP). Perform one forward pass for the given MLP using sigmoid activation.

Answer

Theory: Architecture of MLP

An MLP consists of:

- **Input layer**
- **One or more hidden layers**
- **Output layer**

Each layer performs:

```
z = W*x + b  
a = activation(z)
```

Common activations:

```
sigmoid(x) = 1 / (1 + exp(-x))  
tanh(x)    = (exp(x) - exp(-x)) / (exp(x) + exp(-x))  
ReLU(x)    = max(0, x)
```

Numerical Forward Pass

MLP with:

- Input: $x = [1, 0]$
- Hidden layer weights:

```
W1 = [ [0.2, -0.1],  
       [0.4,  0.3] ]
```

```
b1 = [0.1, -0.2]
```

- Output weights:

```
W2 = [0.5, -0.4]
```

$$b2 = 0.2$$

Activation: sigmoid.

Step 1: Hidden Layer

Compute:

$$z1 = W1 * x + b1$$

$$z1_1 = 0.2 * 1 + (-0.1) * 0 + 0.1 = 0.3$$

$$z1_2 = 0.4 * 1 + 0.3 * 0 - 0.2 = 0.2$$

Apply sigmoid:

$$a1_1 = 1 / (1 + \exp(-0.3)) = 0.5744$$

$$a1_2 = 1 / (1 + \exp(-0.2)) = 0.5498$$

Step 2: Output Layer

Compute:

$$z2 = W2 * a1 + b2$$

$$z2 = 0.5 * 0.5744 + (-0.4) * 0.5498 + 0.2$$

$$z2 = 0.2872 - 0.2199 + 0.2 = 0.2673$$

Output:

$$a2 = \text{sigmoid}(0.2673) = 0.5664$$

Final Output

$$\text{Output} = 0.5664$$

Q3. (Theory + Numerical)

Explain feedforward networks and compute outputs for a ReLU-based network.

Answer

Theory

Feedforward networks move information **forward only** from input $\rightarrow$ hidden $\rightarrow$ output.
No cycles or feedback connections.

General formula:

$$a^{(l)} = \text{activation}(W^{(l)} * a^{(l-1)} + b^{(l)})$$

Numerical Problem

Given:

Input:

$$x = [2, -3]$$

Layer 1:

$$W1 = \begin{bmatrix} 1 & -2 \\ 3 & 1 \end{bmatrix}$$

$$b1 = [1, -1]$$

Activation: ReLU

Output layer:

$$W2 = [2, -1]$$

$$b2 = 0$$

Step 1: Hidden Layer Compute

$$z1 = W1 * x + b1$$

$$z1_1 = 1*2 + (-2)*(-3) + 1 = 2 + 6 + 1 = 9$$

$$z1_2 = 3*2 + 1*(-3) - 1 = 6 - 3 - 1 = 2$$

ReLU:

$$a1_1 = \max(0, 9) = 9$$

$$a1_2 = \max(0, 2) = 2$$

Step 2: Output

$$z2 = W2 * a1 + b2 = 2*9 + (-1)*2 = 18 - 2 = 16$$

Final Output

$$\text{Output} = 16$$

Q4. (Theory + Numerical)

Explain backpropagation and compute gradients for a 1-neuron network using MSE loss.

Answer

Theory: Backpropagation

Backpropagation computes gradients layer-by-layer using:

$$\delta(L) = dL/da(L) * da/dz$$
$$\delta(l) = (W(l+1))^T * \delta(l+1) * da(l)/dz(l)$$

Weight update:

$$W = W - \eta * \delta * a(\text{prev})$$
$$b = b - \eta * \delta$$

Numerical Example

Single neuron:

$$y = \text{sigmoid}(w*x + b)$$

Given:

$$x = 2$$
$$\text{target } t = 1$$
$$w = 0.5$$
$$b = 0$$
$$\text{learning_rate} = 0.1$$

Step 1: Forward Pass

$$z = w*x + b = 1.0$$
$$y = 1/(1 + \exp(-1)) = 0.7311$$

Loss (MSE):

$$L = (t - y)^2 = (1 - 0.7311)^2 = 0.0723$$

Step 2: Gradients

1. dL/dy :

$$dL/dy = -2*(t - y) = -2*(1 - 0.7311) = -0.5378$$

2. dy/dz :

$$dy/dz = y*(1 - y) = 0.7311*(0.2689) = 0.1966$$

3. Chain rule:

$$\delta = dL/dz = dL/dy * dy/dz = -0.5378 * 0.1966 = -0.1057$$

4. Gradients:

$$dL/dw = \delta * x = -0.2114$$

$$dL/db = \delta = -0.1057$$

Step 3: Weight Update

$$w_{\text{new}} = w - \eta * dL/dw = 0.5 - 0.1*(-0.2114) = 0.5211$$

$$b_{\text{new}} = 0 - 0.1*(-0.1057) = 0.0106$$

Final Updated Weights

$$w = 0.5211$$

$$b = 0.0106$$

Q5. (Theory + Numerical)

Explain the effect of multiple hidden layers and train a 2-layer network for one step of gradient descent.

Answer

Theory

Multiple hidden layers allow networks to learn:

- hierarchical features
- complex non-linear mappings
- deep representations

Advantages:

- High expressive power
 - Automatically learn features
 - Universal function approximation
-

Numerical: One training step

Given:

Input:

$$x = [1, 1]$$

Hidden layer:

$$W1 = \begin{bmatrix} 0.1 & 0.2 \\ 0.3 & 0.4 \end{bmatrix}$$

$$b1 = [0, 0]$$

Output:

$$W2 = [0.5, -0.6]$$

$$b2 = 0.2$$

Activation: sigmoid

Target:

$$t = 1$$

Learning rate:

$$\eta = 0.1$$

Step 1: Forward Pass

Hidden:

$$z1_1 = 0.1*1 + 0.2*1 = 0.3$$

$$z1_2 = 0.3*1 + 0.4*1 = 0.7$$

$$a1_1 = \text{sigmoid}(0.3) = 0.5744$$

$$a1_2 = \text{sigmoid}(0.7) = 0.6682$$

Output:

$$z2 = 0.5*0.5744 + (-0.6)*0.6682 + 0.2 = 0.0406$$

$$a2 = \text{sigmoid}(0.0406) = 0.5101$$

Step 2: Output Layer Gradient

$$dL/dy = -2*(t - y) = -2*(1 - 0.5101) = -0.9798$$

$$dy/dz2 = a2*(1 - a2) = 0.5101*0.4899 = 0.2499$$

$$\delta 2 = dL/dz2 = -0.9798*0.2499 = -0.245$$

Gradients:

$$dL/dW2_1 = \delta 2 * a1_1 = -0.245 * 0.5744 = -0.1407$$

$$dL/dW2_2 = \delta 2 * a1_2 = -0.245 * 0.6682 = -0.1637$$

$$dL/db2 = \delta 2 = -0.245$$

Step 3: Hidden Layer Gradient

$$\delta 1_i = W2_i * \delta 2 * a1_i * (1 - a1_i)$$

For neuron 1:

$$\delta 1_1 = 0.5 * (-0.245) * 0.5744 * (1 - 0.5744)$$

$$\delta 1_1 = -0.5 * 0.245 * 0.2444 = -0.0299$$

For neuron 2:

$$\delta 1_2 = (-0.6) * (-0.245) * 0.6682 * (1 - 0.6682)$$

$$\delta 1_2 = 0.6 * 0.245 * 0.2214 = 0.0327$$

Gradients:

$$dL/dW1_ij = \delta 1_i * x_j$$

Thus:

$$dL/dW1 =$$

$$\begin{bmatrix} [-0.0299, -0.0299], \\ [0.0327, 0.0327] \end{bmatrix}$$

Step 4: Weight Updates

Example:

$$W2_1 = 0.5 - 0.1 * (-0.1407) = 0.5141$$

You can similarly update all weights.

★ **Q1. (Theory + Numerical)

Minimax Algorithm, Game Tree, and Adversarial Decision Making**

Question

- (a) Explain the minimax algorithm for two-player zero-sum games using a game tree.
(b) Given the following game tree (MAX at root), compute the minimax value:



Answer

Theory

A **two-player zero-sum game** involves:

- **MAX** → tries to maximize utility
- **MIN** → tries to minimize MAX's utility

Minimax chooses:

Utility(state) = max over MIN's responses of min over MAX's responses

...

Game tree levels:

- MAX level → choose maximum child
- MIN level → choose minimum child
- Continue until leaf utilities.

Numerical Solution

Level 1 (MIN nodes evaluate):

$A = \min(3, 5) = 3$
 $B = \min(2, 9) = 2$
 $C = \min(1, 4) = 1$

Now MAX chooses:

$\text{Value} = \max(A, B, C) = \max(3, 2, 1) = 3$

Final Minimax Value

Minimax result = 3

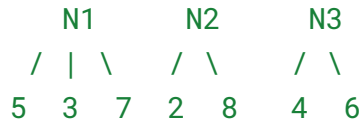
★ **Q2. (Theory + Numerical)

Alpha-Beta Pruning and Pruning Efficiency**

Question

- Explain alpha-beta pruning. Why does it improve minimax efficiency?
- Apply alpha-beta pruning on the following tree and show which branches are pruned:

MAX
 / | \



Answer

Theory: Alpha-Beta Pruning

Two parameters:

`alpha` = best value MAX can guarantee

`beta` = best value MIN can guarantee

Prune a branch when:

`alpha >= beta`

Meaning the opponent will never allow us to reach that branch.

Pruning reduces time from:

`$O(b^d)$  to  $O(b^{(d/2)})$`

Numerical Walkthrough

Explore N1

Children: 5, 3, 7 → MIN level:

`Value(N1) = 3`

`alpha = 3`

Explore N2

Children: 2, 8 → MIN evaluates:

First child:

$\beta = 2$

Now compare:

$\alpha = 3$

$\beta = 2$

$\alpha \geq \beta \rightarrow$ prune second child of N2 (value 8)

Thus:

$\text{Value}(N2) = 2$

Explore N3

Children: 4, 6 (MIN level)

First child:

$\beta = 4$

Compare:

$\alpha = 3$

$\beta = 4 \rightarrow$ continue

Second child:

$\beta = \min(4, 6) = 4$

$\text{Value}(N3) = 4$

No pruning here.

Final MAX selection

$\text{MAX}(3, 2, 4) = 4$

Pruned Node

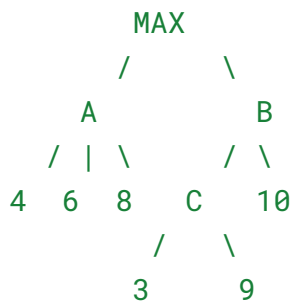
Value 8 under N2 is pruned.

★ **Q3. (Theory + Numerical)

Expectimax Algorithm and Stochastic Game Evaluation**

Question

- (a) Explain how the expectimax algorithm differs from minimax.
- (b) Compute the expectimax value of the stochastic game tree:



Probability at C: $P(3)=0.7$, $P(9)=0.3$

Answer

Theory

Minimax:

MIN chooses min-value child.

Expectimax:

Chance nodes choose expected value:
 $\text{Expected} = \sum p_i * \text{value}_i$

Used when:

- Opponent is *random*, not adversarial (e.g., dice games, random transitions).

Numerical Solution

Node A (MAX chooses):

$$A = \max(4, 6, 8) = 8$$

Node C (Chance node):

$$\text{Value}(C) = 0.7*3 + 0.3*9 = 2.1 + 2.7 = 4.8$$

Node B (MAX chooses):

$$B = \max(C, 10) = \max(4.8, 10) = 10$$

Root MAX:

$$\text{Value} = \max(A, B) = \max(8, 10) = 10$$

Final Expectimax Value

$$\text{Expectimax result} = 10$$

★ **Q4. (Theory + Numerical)

Markov Chains, State Transitions, Steady-State Probabilities**

Question

- (a) What is a Markov chain? Explain state transitions and the Markov property.
(b) Given transition matrix:

$$P = \begin{bmatrix} 0.6 & 0.4 \\ 0.2 & 0.8 \end{bmatrix}$$

Find the **steady-state distribution** π such that:

$$\begin{aligned}\pi P &= \pi \\ \pi_1 + \pi_2 &= 1\end{aligned}$$

Answer

Theory

A **Markov chain** is a stochastic system with:

- finite states
- transition probabilities
- **Markov property**: next state depends only on current state

Transition equation:

$$\pi' = \pi P$$

Steady state satisfies:

$$\pi = \pi P$$

Numerical Solution

Solve:

$$\pi_1 = 0.6\pi_1 + 0.2\pi_2$$

$$\pi_2 = 0.4\pi_1 + 0.8\pi_2$$

$$\pi_1 + \pi_2 = 1$$

Use first equation:

$$\pi_1 = 0.6\pi_1 + 0.2(1 - \pi_1)$$

$$\pi_1 = 0.6\pi_1 + 0.2 - 0.2\pi_1$$

$$\pi_1 - 0.4\pi_1 = 0.2$$

$$0.6\pi_1 = 0.2$$

$$\pi_1 = 0.3333$$

$$\pi_2 = 1 - \pi_1 = 0.6667$$

Final Steady-State Distribution

$$\pi = [0.3333, 0.6667]$$

**Q5. (Theory + Numerical)

MDPs, Bellman Equations, Value Iteration, and Optimal Policies**

Question

(a) Explain states, actions, transitions, and rewards in MDPs. Write Bellman optimality equations.

(b) Perform one iteration of **value iteration** for the MDP:

States: $S = \{A, B\}$

Actions: a_1 only

Transition model:

$$T(A \rightarrow A) = 0.7$$

$$T(A \rightarrow B) = 0.3$$

$$T(B \rightarrow A) = 0.4$$

$$T(B \rightarrow B) = 0.6$$

Rewards:

$$R(A) = 5$$

$$R(B) = 2$$

Discount:

$$\gamma = 0.9$$

Initial values:

$$V_0(A) = 0$$

$$V_0(B) = 0$$

Answer

Theory

Components of an MDP:

States: S

Actions: A

Transition model: $T(s, a, s')$

Reward: $R(s)$

Policy: $\pi(s)$

Bellman optimality:

$$V^*(s) = \max_a \sum_{s'} T(s, a, s') [R(s) + \gamma V^*(s')]$$

Value iteration update:

$$V_{\{k+1\}}(s) = \max_a \sum_{s'} T(s, a, s') [R(s) + \gamma V_k(s')]$$

Numerical Calculation

Only one action $\Rightarrow$ no max needed.

Update A:

$$V1(A) = 0.7*(5 + 0.9*0) + 0.3*(5 + 0.9*0)$$

$$V1(A) = 5$$

Update B:

$$V1(B) = 0.4*(2 + 0.9*0) + 0.6*(2 + 0.9*0)$$

$$V1(B) = 2$$

Values After One Iteration

$$V1(A) = 5$$

$$V1(B) = 2$$

=====

=====

★ Q1. RL Basics + Numerical (Q-Learning Update)

=====

=====

Question

- (a) Explain Reinforcement Learning (RL) with components: states, actions, rewards, policy, value.
- (b) Differentiate Q-Learning and TD(0).
- (c) For the transition below, compute the updated Q-value using Q-Learning:

State: $s = S1$

Action: $a = A1$

Reward: $r = 4$

Next state: $s' = S2$

Discount factor: $\gamma = 0.9$

Learning rate: $\alpha = 0.5$

Current $Q(S1, A1) = 6$

Q-values in S2: $Q(S2, A1)=5$, $Q(S2, A2)=8$

Answer

(a) Introduction to RL

RL involves an agent interacting with an environment.

Core components:

State: s

Action: a

Reward: r

Policy: $\pi(s)$

Value function: $V(s)$

Action-value: $Q(s, a)$

Goal: maximize cumulative reward:

$$G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots$$

(b) Q-Learning vs TD(0)

Q-Learning (off-policy)

Uses greedy target:

$$\text{Target} = r + \gamma \max_{a'} Q(s', a')$$

TD(0) updates value using:

$$V(s) \leftarrow V(s) + \alpha [r + \gamma V(s') - V(s)]$$

TD(0) learns from single-step bootstrapping; Q-Learning learns optimal policy even if behavior is exploratory.

(c) Q-Learning Numerical Update

Formula:

$$Q_{\text{new}} = Q_{\text{old}} + \alpha * (r + \gamma \max_{a'} Q(s', a') - Q_{\text{old}})$$

Compute max:

$$\max_{a'} Q(S2, a') = 8$$

Compute target:

$$\text{Target} = 4 + 0.9 * 8 = 4 + 7.2 = 11.2$$

Update:

$$Q_{\text{new}} = 6 + 0.5 * (11.2 - 6)$$

$$Q_{\text{new}} = 6 + 0.5 * 5.2 = 6 + 2.6 = 8.6$$

Final Answer

$$Q_{\text{new}}(S1, A1) = 8.6$$

=====

=====

★ Q2. SARSA + Exploration vs Exploitation (Numerical + Theory)

=====

=====

Question

- (a) Explain SARSA as an on-policy TD control method.
- (b) Explain the exploration–exploitation trade-off.
- (c) Compute SARSA update:

Current state: S1

Action: A1

Reward: $r = 2$

Next state: S2

Next chosen action (on-policy): A2

$\gamma = 0.8$

$\alpha = 0.4$

$Q(S1, A1) = 5$

$Q(S2, A2) = 7$

Answer

(a) SARSA Explanation

SARSA learns from the sequence:

(s, a, r, s', a')

Update rule:

$$Q(s,a) \leftarrow Q(s,a) + \alpha [r + \gamma Q(s',a') - Q(s,a)]$$

SARSA is **on-policy**, meaning the same policy is used for acting and learning.

(b) Exploration vs Exploitation

- **Exploration:** try new actions
- **Exploitation:** use the best-known action
- Balanced using ϵ -greedy:

With probability $\epsilon \rightarrow$ random action

With probability $1-\epsilon \rightarrow$ greedy action

(c) SARSA Numeric Update

Formula:

$$Q_{\text{new}} = Q_{\text{old}} + \alpha (r + \gamma Q(s',a') - Q_{\text{old}})$$

Substitute:

$$\text{Target} = 2 + 0.8 \cdot 7 = 2 + 5.6 = 7.6$$

$$Q_{\text{new}} = 5 + 0.4 \cdot (7.6 - 5) = 5 + 0.4 \cdot 2.6 = 5 + 1.04 = 6.04$$

Final Answer

$$Q_{\text{new}}(S1, A1) = 6.04$$

★ Q3. Discount Factor γ + Policy Iteration

Question

- (a) Explain the discount factor γ and its effect on long-term vs short-term rewards.
- (b) Describe Policy Evaluation and Policy Improvement steps in Policy Iteration.
- (c) Given the MDP:

States: A, B

Rewards: $R(A)=5$, $R(B)=3$

Transitions are deterministic:

$A \rightarrow B$

$B \rightarrow A$

Policy: always move to next state

$\gamma = 0.9$

Initial values: $V_0(A)=0$, $V_0(B)=0$

Perform **one full iteration of Policy Evaluation**.

Answer

(a) Discount Factor γ

$$0 < \gamma < 1$$

Controls importance of future rewards.

If γ is small:

Immediate rewards dominate.

If γ is large:

Long-term planning is preferred.

(b) Policy Iteration Steps

Policy Evaluation

$$V_{k+1}(s) = R(s) + \gamma V_k(\pi(s))$$

Policy Improvement

$$\pi_{\text{new}}(s) = \operatorname{argmax}_a \sum_{s'} T(s, a, s') [R(s) + \gamma V(s')]$$

(c) Numerical Policy Evaluation

Policy:

$A \rightarrow B$

$B \rightarrow A$

Update A:

$$V_1(A) = R(A) + \gamma V_0(B)$$

$$V_1(A) = 5 + 0.9 \cdot 0 = 5$$

Update B:

$$V_1(B) = R(B) + \gamma V_0(A)$$

$$V_1(B) = 3 + 0.9 \cdot 0 = 3$$

Final Evaluated Values

$V_1(A) = 5$

$V_1(B) = 3$

=====

=====

★ Q4. Value Iteration + Bellman Optimality (Numerical + Theory)

=====

=====

Question

- (a) Write the Bellman optimality equation for $V^*(s)$.
- (b) Explain Value Iteration.
- (c) Use value iteration for one iteration in the MDP:

States: A, B

Action: a only

Rewards: $R(A)=2$, $R(B)=1$

Transitions:

$A \rightarrow A: 0.4$, $A \rightarrow B: 0.6$

$B \rightarrow A: 0.5$, $B \rightarrow B: 0.5$

$\gamma = 0.9$

Initial: $V_0(A)=0$, $V_0(B)=0$

Answer

(a) Bellman Optimality

$$V^*(s) = \max_a \sum_{s'} T(s, a, s') [R(s) + \gamma V^*(s')]$$

(b) Value Iteration

Iterative update:

$$V_{k+1}(s) = \max_a \sum_{s'} T(s, a, s') [R(s) + \gamma V_k(s')]$$

Converges to optimal V^* .

(c) Numerical Update

Update A:

$$V_1(A) = 0.4*(2 + 0.9*0) + 0.6*(2 + 0.9*0)$$

$$V_1(A) = 2$$

Update B:

$$V_1(B) = 0.5*(1 + 0.9*0) + 0.5*(1 + 0.9*0)$$

$$V_1(B) = 1$$

Final Values

$$V_1(A) = 2$$

$$V_1(B) = 1$$

=====

=====

★ Q5. Actor–Critic Methods (Theory + Numerical TD Error)

=====

=====

Question

- (a) Explain the architecture of Actor–Critic methods.
- (b) List advantages over pure value-based or policy-based methods.
- (c) Compute the TD error δ and critic update for:

$V(s) = 4$
 $V(s') = 5$
Reward $r = 3$
 $\gamma = 0.9$
 $\alpha = 0.2$

Answer

(a) Actor–Critic Architecture

Two components:

Actor

$\pi_{\theta}(a|s)$

Improves policy using gradient:

$\theta \leftarrow \theta + \alpha * \delta * \nabla_{\theta} \log \pi_{\theta}(a|s)$

Critic

Approximates value:

$$V_w(s)$$

Updated using TD error.

(b) Advantages

- Lower variance than policy gradient
 - Can handle continuous actions
 - Faster convergence
 - Combines advantages of value + policy methods
-

(c) TD Error and Update

TD error:

$$\delta = r + \gamma V(s') - V(s)$$

$$\delta = 3 + 0.9 \cdot 5 - 4$$

$$\delta = 3 + 4.5 - 4 = 3.5$$

Critic update:

$$V_{\text{new}}(s) = V(s) + \alpha * \delta$$

$$V_{\text{new}}(s) = 4 + 0.2 \cdot 3.5$$

$$V_{\text{new}}(s) = 4 + 0.7 = 4.7$$

Final Answers

$$\delta = 3.5$$

$$V_{\text{new}}(s) = 4.7$$